

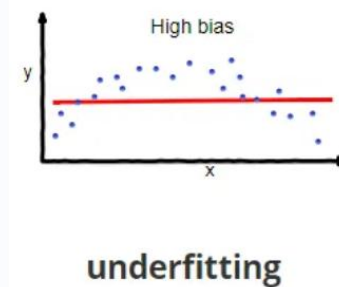


Data Science Lecture 5

Prof. Dr. Rasha Ismail
Dr. Eman Amin

What is Bias?

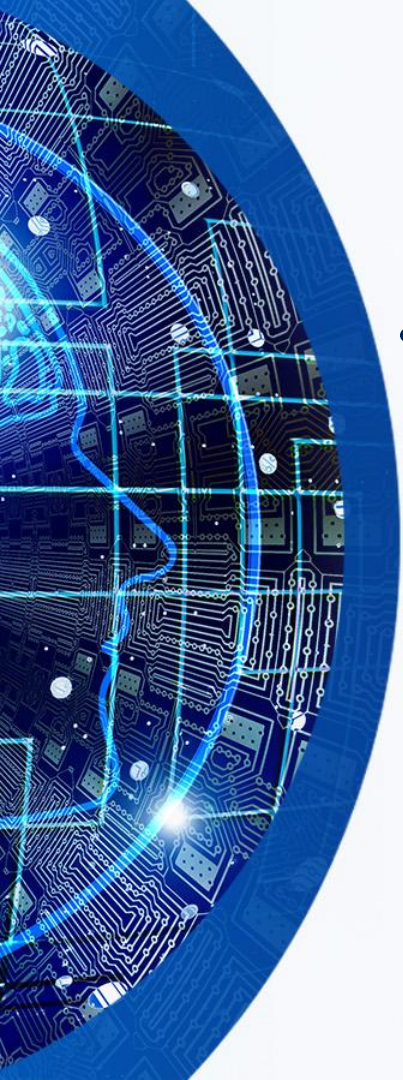
- **Bias** can be defined as the error between model predictions and the actual results. Essentially, it describes how well the model captures in **the training data set**.
 - A model which doesn't capture the trends in training data set well is said to show high bias.
 - A model with low bias resembles the trends in the data set .





What is Bias?

- **Characteristics of a high bias model include:**
 - **Underfitting:** The model fails to capture the underlying patterns in the data. It performs poorly on both the training and testing data.
 - **Simple Model:** The model is often too simple, with fewer parameters or a less complex structure. This simplicity limits its ability to learn relationships.
 - **High Training Error:** The model has a high error rate on the training data itself, indicating that it's not learning effectively.
 - **Poor Generalization:** The model performs poorly on new, unseen data.

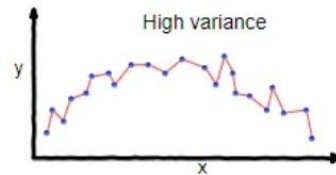


What is Bias?

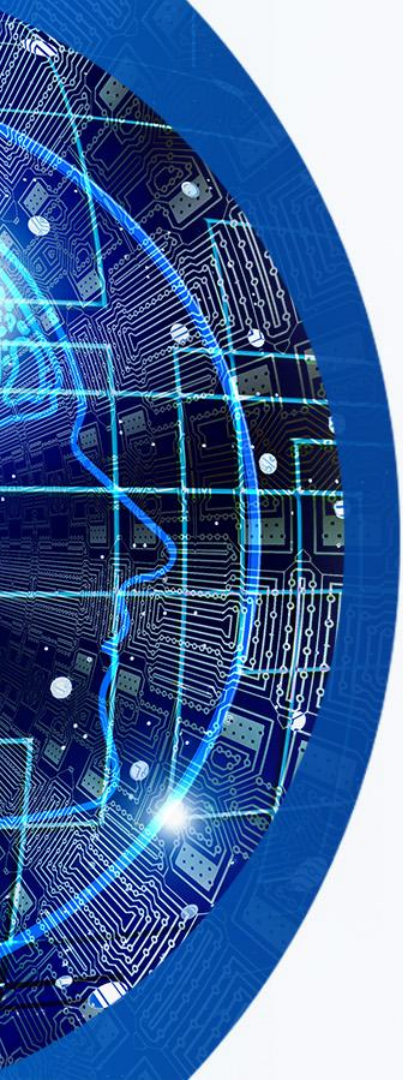
- To address high bias, consider the following strategies:
 - **Increase Model Complexity:** Use more complex models with more parameters or features.
 - **Gather More Data:** A larger dataset can help the model learn more complex patterns.
 - **Feature Engineering:** Create more relevant features that can improve the model's ability to capture underlying relationships.

What is Variance?

- **Variance** is the variability of model prediction for a given data point or a value which tells us spread of our data.
- Model with high variance pays a lot of attention to training data and does not generalize on the data which it hasn't seen before.
 - As a result, such models perform very well on training data but has high error rates on test data.

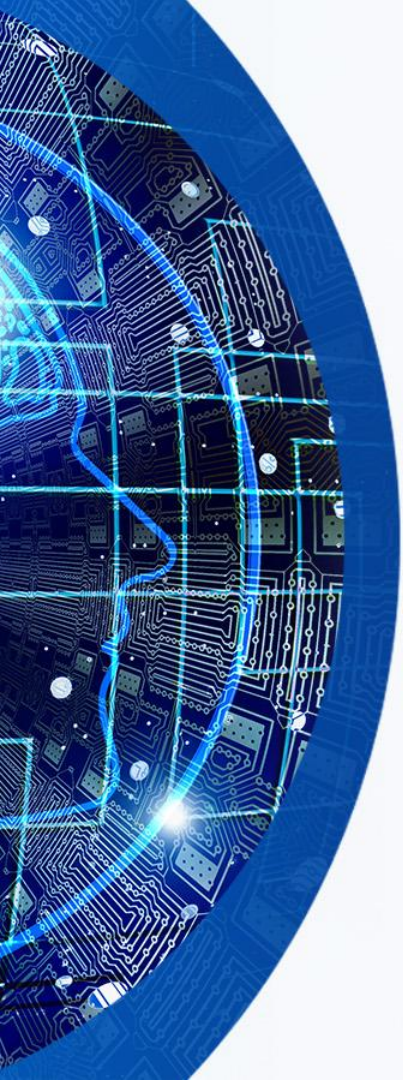


overfitting



What is Variance?

- **Characteristics of a high variance model include:**
 - **Overfitting:** The model fits the training data too closely, capturing noise and outliers rather than the underlying patterns.
 - **Poor Generalization:** The model performs well on the training data but poorly on new, unseen data.
 - **Complex Model:** High variance models are often complex, with many parameters or a highly flexible structure.
 - **Low Training Error:** The model has a very low error rate on the training data, but this is often misleading.
 - **High Test Error:** The model performs poorly on the testing data, indicating its inability to generalize.



What is Variance?

- To address high variance, consider the following strategies:
 - **Regularization Techniques**
 - **Feature Selection:** Remove irrelevant or redundant features to reduce the model's complexity.
 - **Model Simplification:** Use simpler models with fewer parameters.

Bias and Variance

- Centre of the target is a model that perfectly predicts correct values. As we move away from the bulls-eye our predictions become get worse and worse.
- When bias is high, focal point of group of predicted function lie far from the true function. Whereas, when variance is high, functions from the group of predicted ones, differ much from one another.
- Clearly, low bias and low-variance is the most desired outcome while high-bias and high-variance the least.

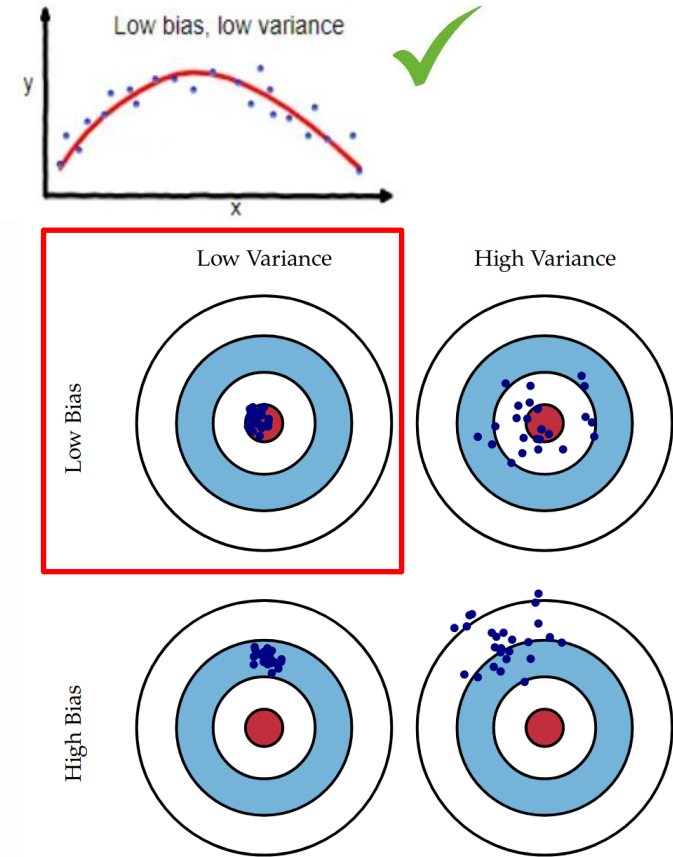
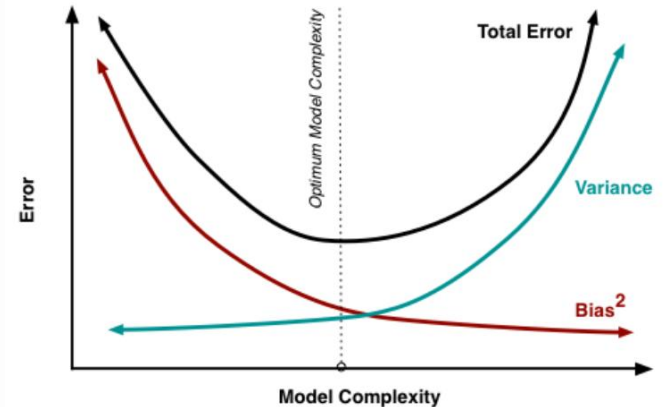


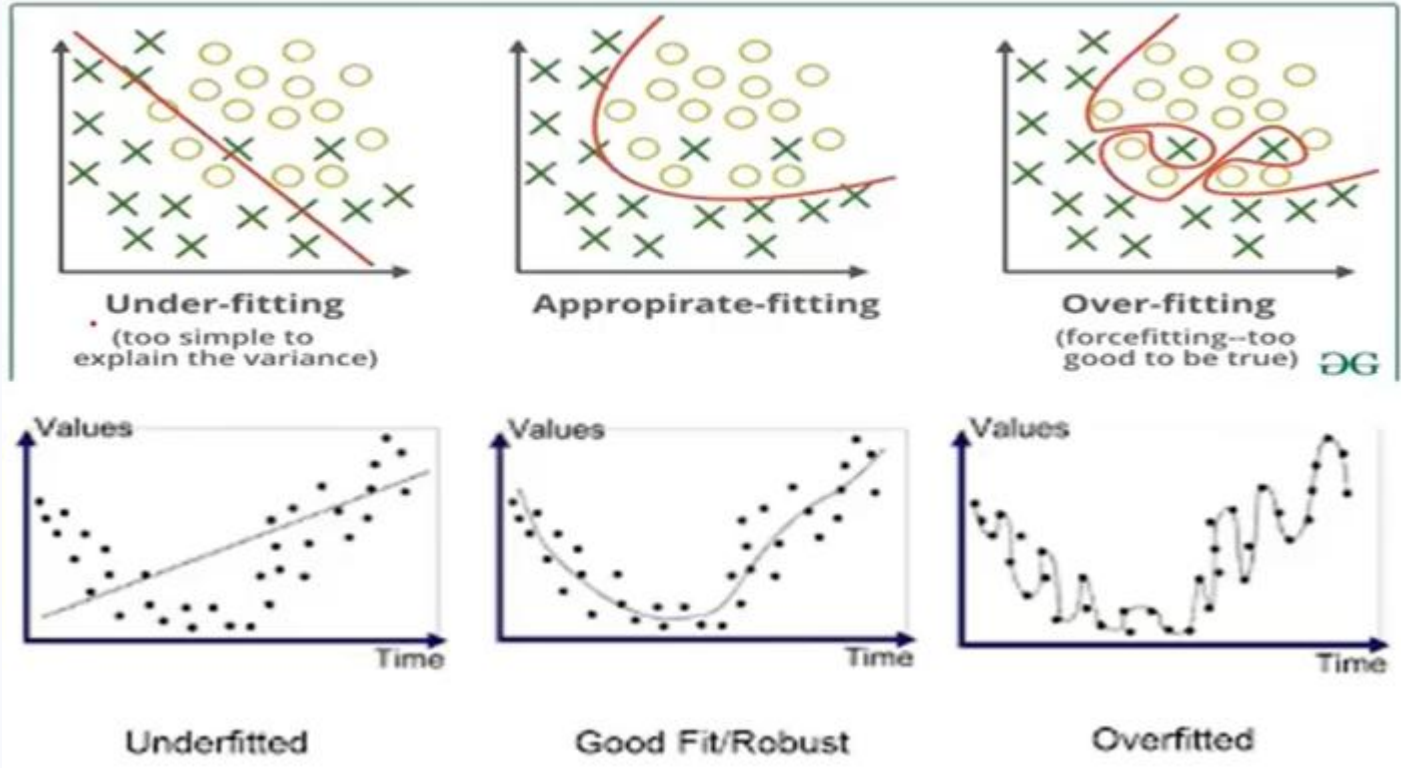
Fig. 1 Graphical illustration of bias and variance.

Bias and Variance Tradeoff

- The graph shows how training and testing error change as model complexity increases.
- As model complexity increases, bias decreases (the model becomes more flexible and can fit the data better), but variance increases (the model becomes more sensitive to noise and random fluctuations in the training data).
- The goal is to find the balance spot where the model has low bias and low variance, resulting in high accuracy on both training and testing data.

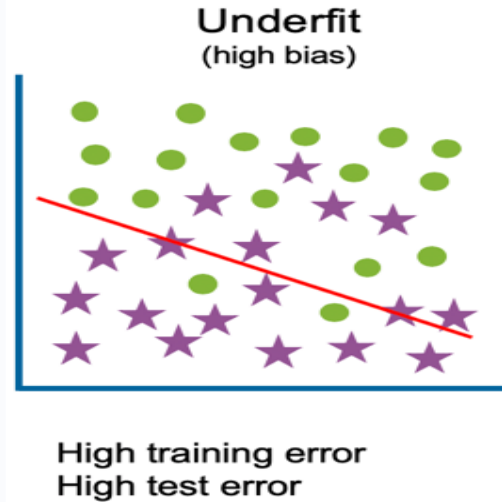


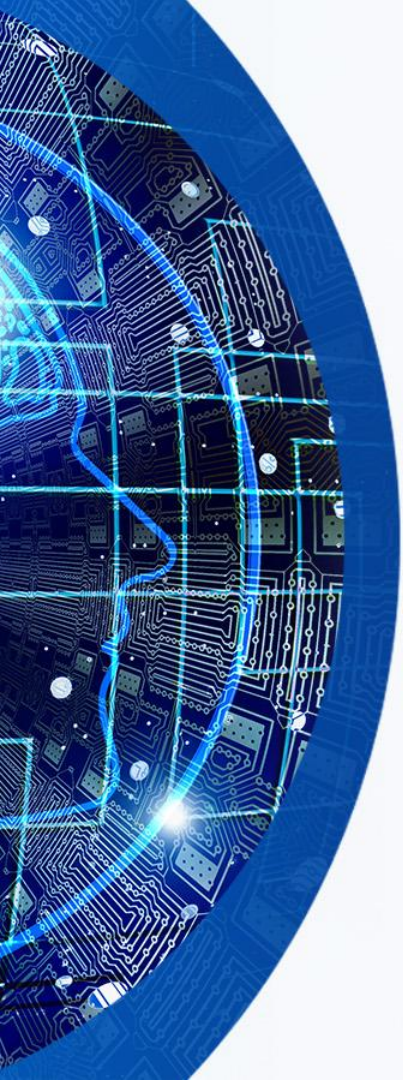
Examples



What is Underfitting

- **Underfitting**: refers to a model that can neither model the training data nor generalize to new data.
- An underfit model is not suitable model and will be obvious as it will have **poor performance** on the training data.



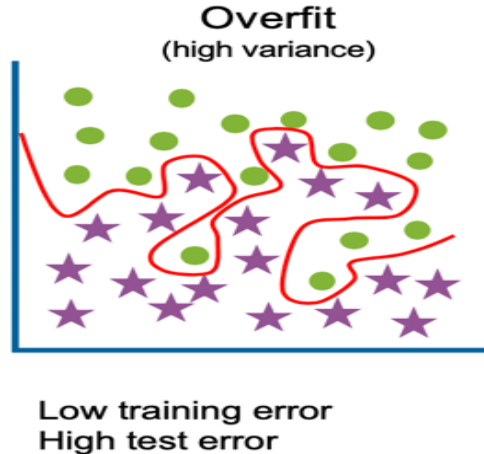


What is Underfitting

- What causes underfit?
 - Model capacity is too small to fit the training dataset as well as generalize to new dataset.
- Solution
 - Increase the capacity of the model
 - Examples:
 - Increase number of layers, neurons in each layer, etc.
- Result:
 - Lower Bias
 - ***Underfit* → *Good Fit*?**

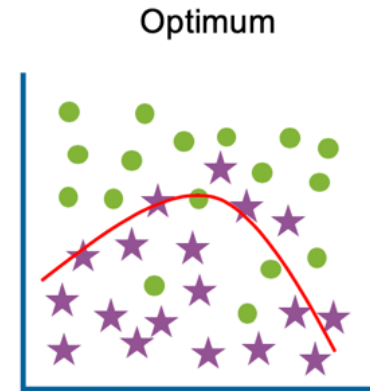
What is Overfitting?

- **Overfitting:** refers to a model that fit the training data too well.
- Overfitting happens when a model learns **the detail and noise in the training data** to the extent that it **negatively impacts the performance** of the model on new data.



Brief

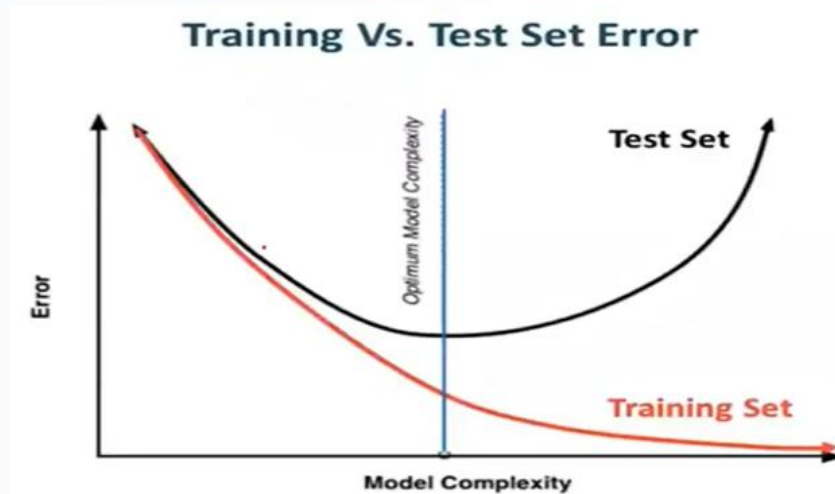
- **Overfitting:** Good performance on the training data, poor generalization to other data.
- **Underfitting:** poor performance on the training data and poor generalization to other data
- **Optimal\ Good Fit:** Good performance on the training data, Good generalization to other data.

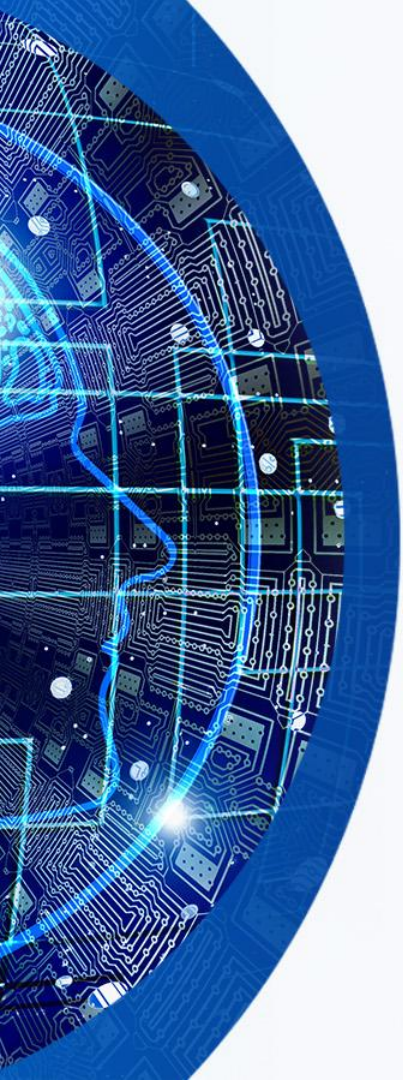


Low training error
Low test error

Problem of Overfitting

- Overfitting occurs when the model performs well on the training data but does not perform accurately in the testing set. The model memorizes the data patterns in the training dataset but fails to generalize to unseen examples.
- Overfitting problem can be solved using Regularization Techniques



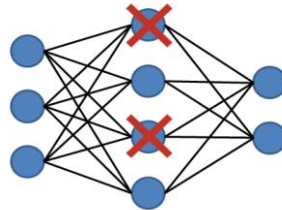


What is Regularization?

- Regularization consists of different techniques and methods used to **address the issue of over-fitting** by **reducing the generalization error**.
- This in turn **improves the model's performance on the unseen data as well**.
- Different Regularization techniques in Deep Learning:
 - **Dropout**
 - **Data Augmentation**
 - **Early Stopping**

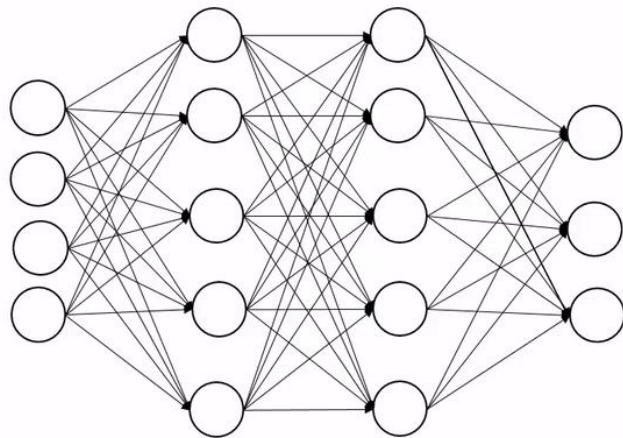
Regularization: Dropout

- A dropout is a regularization approach that prevents overfitting by ensuring that no neurons are co-dependent with one another.
- “Dropout” refers to the practice of disregarding certain neurons in a layer at random during training. The network becomes more adaptable and performs better on unseen data which improves model Generalization.
- At every training step, every neuron has a probability p of being temporarily “dropped out”, meaning it will be entirely ignored during this training step, but it may be active during the next step.



Regularization: Dropout

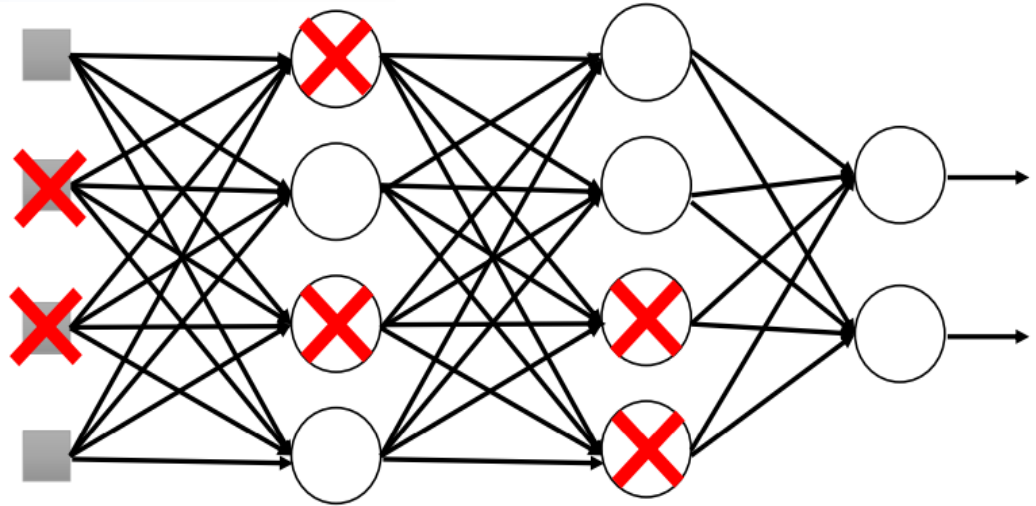
- How it works
 - Randomly selected neurons are ignored during each training step.
 - Dropped neurons don't have effect on next layers.
 - Dropped neurons are not updated in backward training.
- Questions:
 - What're the ideas?
 - Why dropout help to reduce overfit?
 - Dropout randomly "turns off" some neurons during training. This prevents them from becoming too reliant on each other. When a neuron is turned off, others have to step up and contribute, making the network more robust and less prone to overfitting.



Regularization: Dropout

- A good value for dropout probability in a hidden layer is between 0.5 and 0.8.

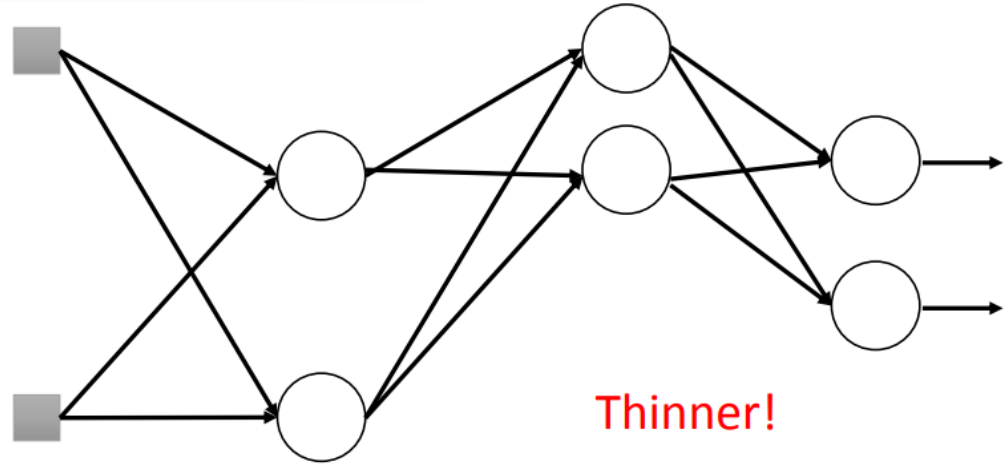
Training:



- **Each time before computing the gradients**
 - Each neuron has $p\%$ to dropout

Regularization: Dropout

Training:

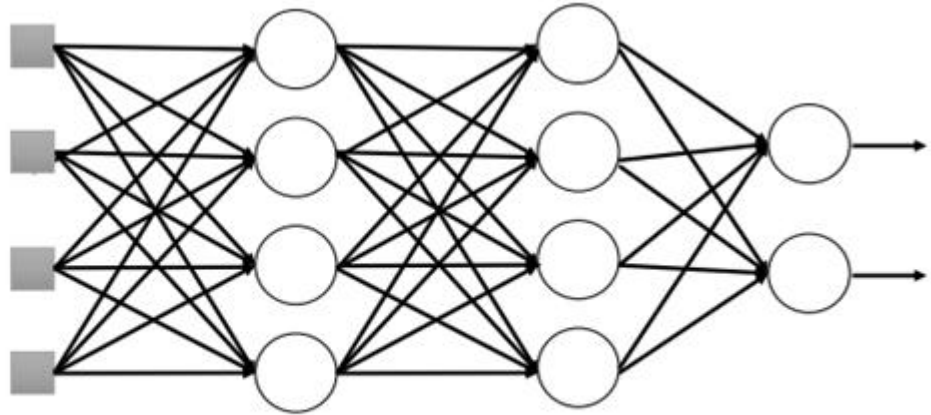


- Each time before computing the gradients
 - Each neuron has $p\%$ to dropout
 - ➡ **The structure of the network is changed.**
 - Using the new network for training

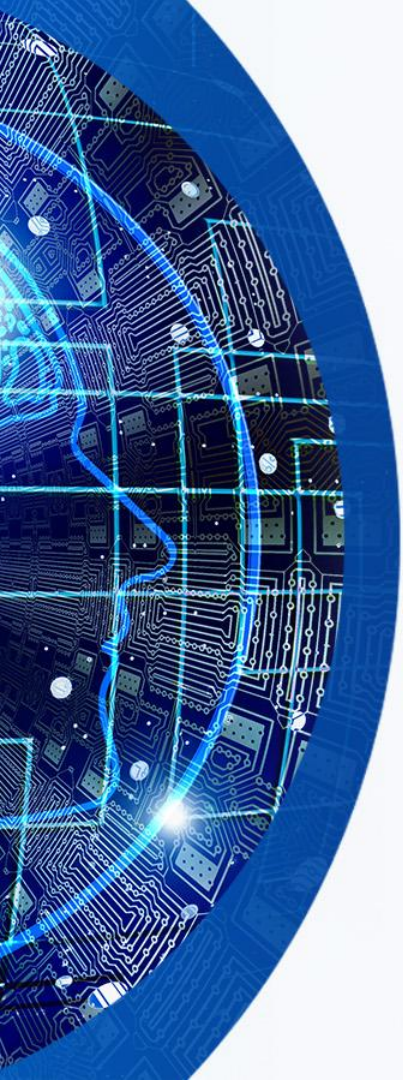
Regularization: Dropout

- During the testing, we do not use a dropout layer. This means that all the units are considered during the prediction step.

Testing:



➤ No dropout

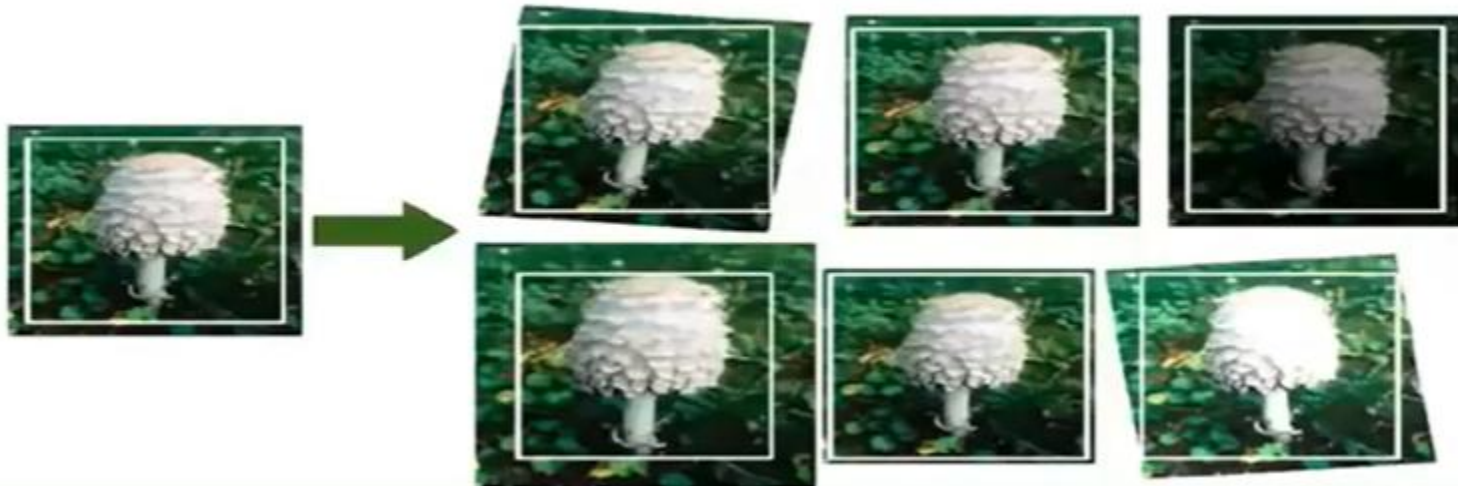


Regularization: Data Augmentation

- **Data augmentation** is a set of techniques to artificially increase the amount of data by generating new data points from existing data.
- This includes adding minor alterations to data or using machine learning models to generate new data points in the latent space of original data to amplify the dataset.
- **Example:** Common ways of obtaining new (image) data include:
 - Flipping each image horizontally or vertically (won't work for digit recognition, but might for other tasks)
 - Shifting each pixel a little to the left or right
 - Rotating the images a little
 - Adding noise to the image

Regularization: Data Augmentation

- **Example:** Common ways of obtaining new (image) data consists of generating new training (rotating, resizing, flipping and cropping) instances from existing ones, artificially boosting the size of the training set.
- This will reduce overfitting, making this a regularization technique. The trick is to generate realistic training instances.



Regularization: Data Augmentation



**Image
Augmentation**



Data Augmentation



Original Image



De-texturized



De-colored



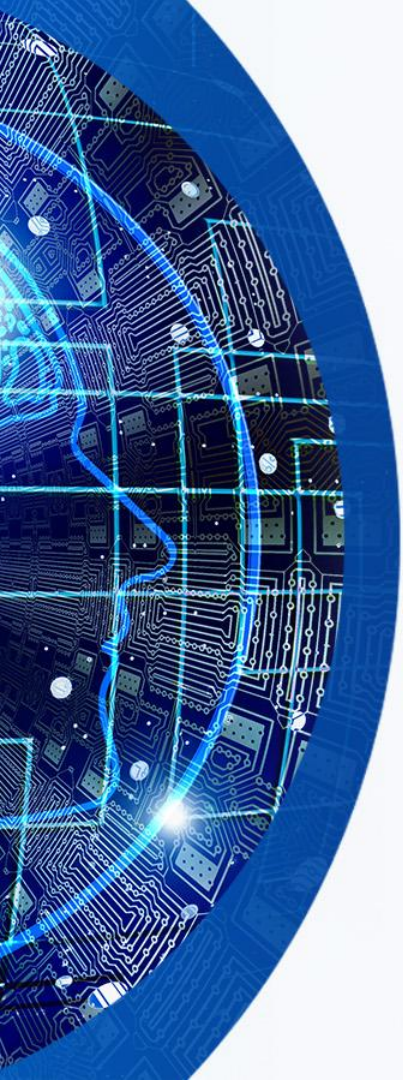
Edge Enhanced



Salient Edge Map



Flip/Rotate

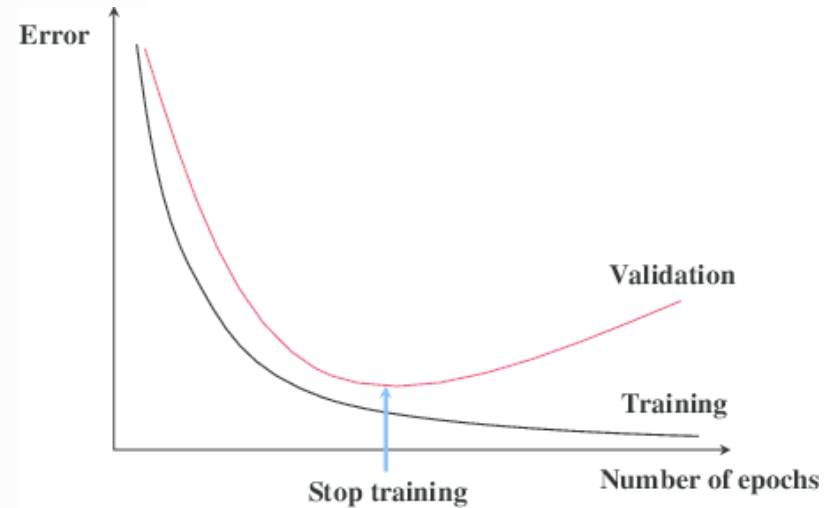


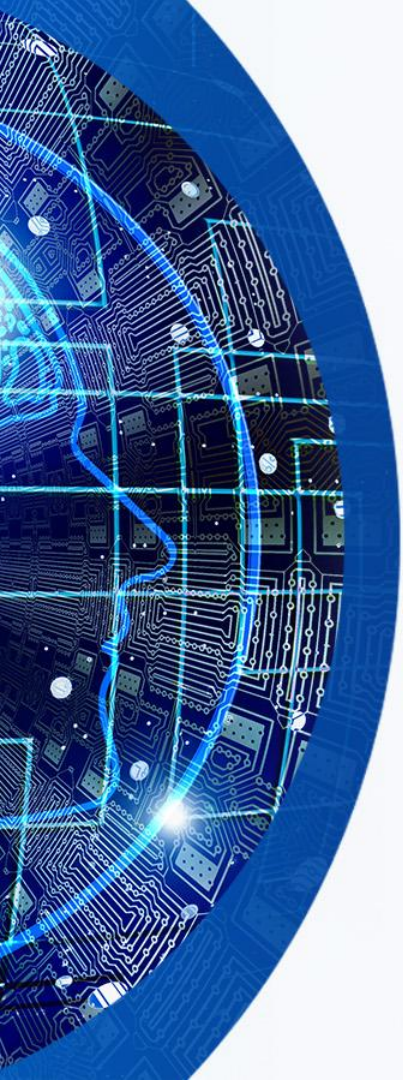
Regularization: Early Stopping

- **Early stopping** is a regularization technique that helps prevent overfitting by monitoring the model's performance on a validation set during training.
- The idea behind early stopping is to prevent overfitting by stopping the training process at the optimal point.
- Overfitting occurs when a model becomes too complex and starts to memorize the training data, including noise and outliers. This leads to poor performance on new, unseen data
- The challenge is to train the network long enough that it can learn the mapping from inputs to outputs, but not training the model so long that it overfits the training data.

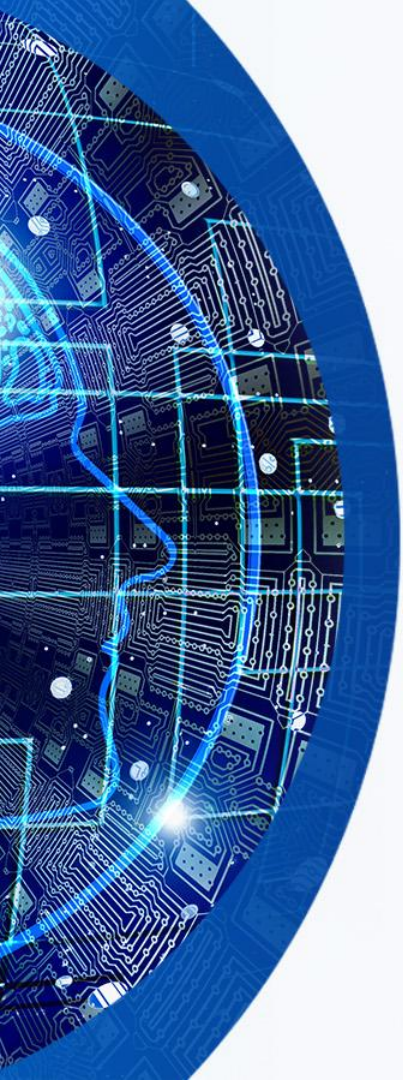
Regularization: Early Stopping

- **How it works:**
 1. **Split the Data:** Divide the dataset into training and validation sets.
 2. **Train the Model:** Train the model on the training set for a specified number of epochs.
 3. **Evaluate on Validation Set:** After each epoch, evaluate the model's performance on the validation set.
 4. **Stop Training:** If the validation performance doesn't improve for a certain number of consecutive epochs, stop the training process.



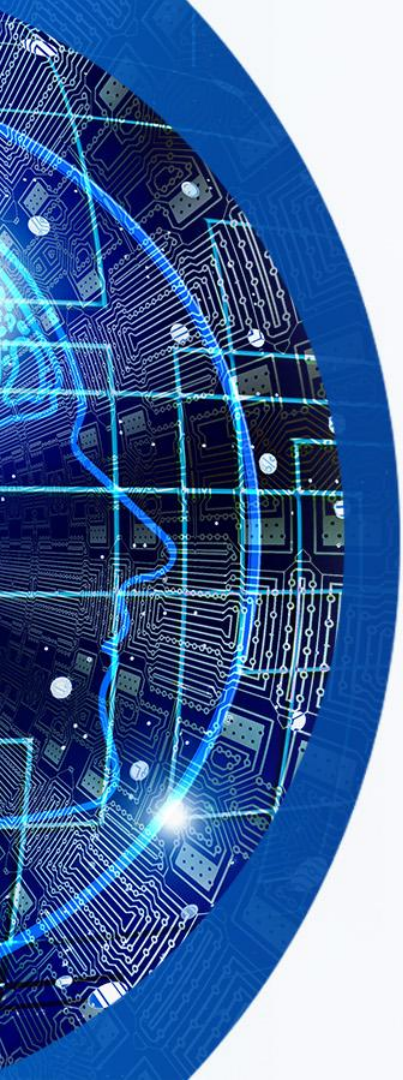


Deep Learning Models (Convolutional Neural Networks)



What is CNNs?

- Convolutional Neural Networks or CNN, the heart of **Computer Vision**.
- **Computer Vision** is one of the most influential fields of application in AI/ML. When machines replicate the complexities of human vision and understanding of the surrounding around them.
- All sorts of **visual data** like images, videos, etc., are used to work on object detection, image segmentation, classification, etc.

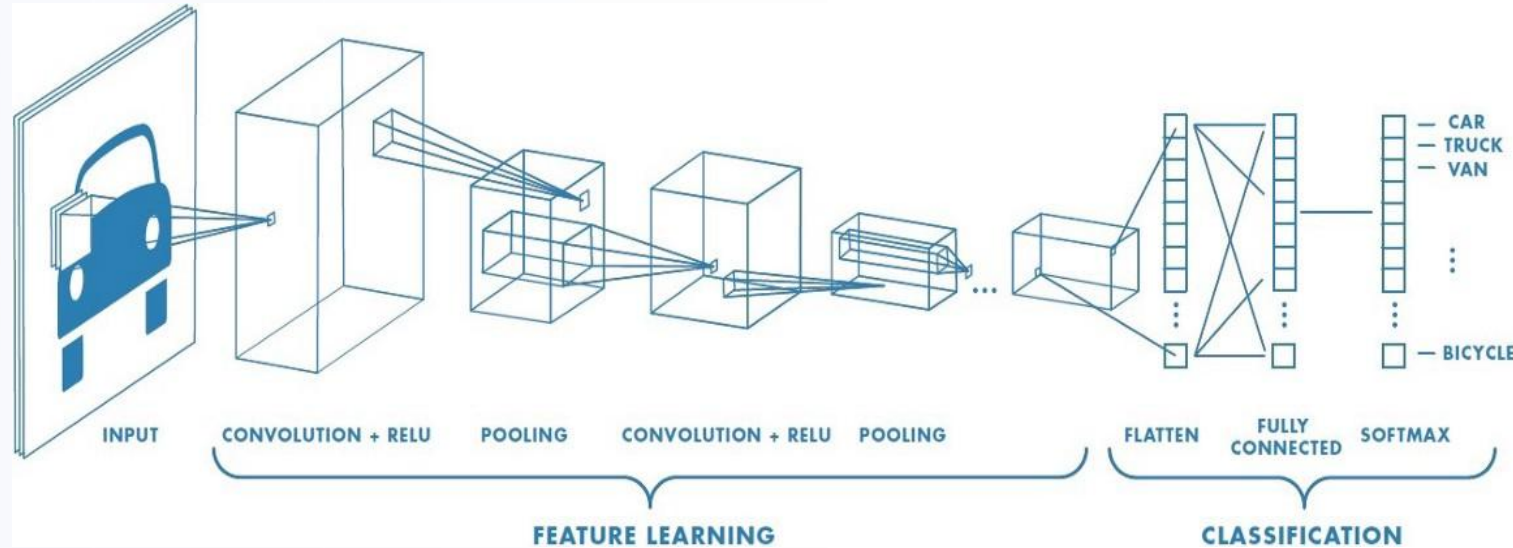


What is CNNs?

- Since image data is very different from tabular data, a special kind of neural network is used to deal with its complexities and derive insights from it, **known as Convolutional Neural Networks.**
- A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other.

What is CNNs?

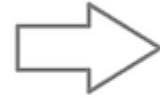
- A CNN example to classify transportations.



Why CNNs over Feed-Forward Neural Nets?

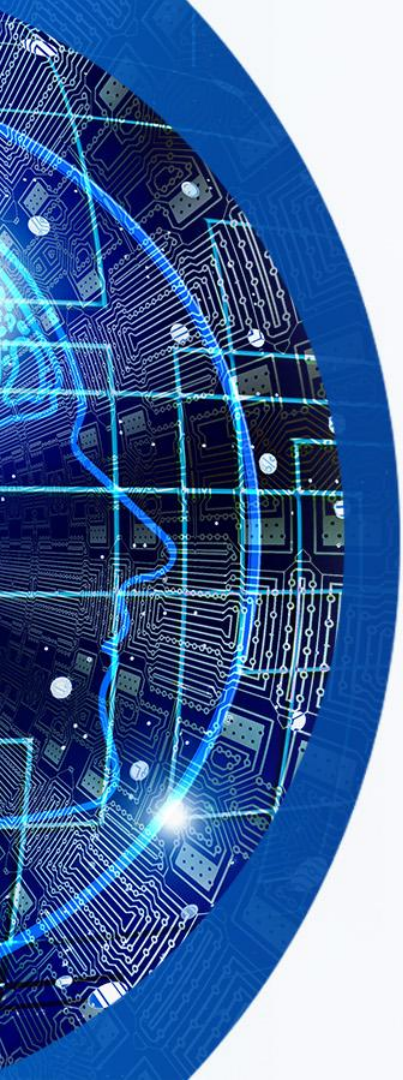
- A computer sees an image as a matrix of pixel values, right? So why not just flatten the image (e.g. 3x3 image matrix into a 9x1 vector) and feed it to a neural network for classification purposes?
- **So, the question is: Can we just flatten the image into one long 1D matrix?**

1	1	0
4	2	1
0	2	1



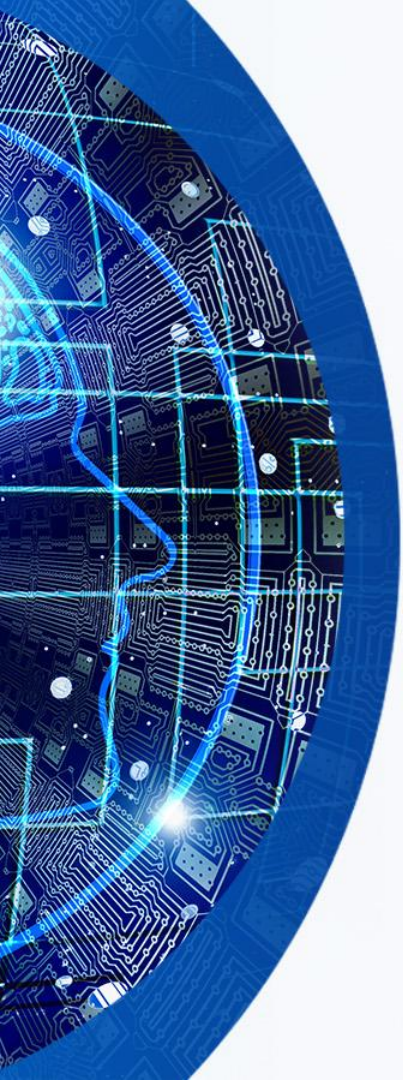
1
1
0
4
2
1
0
2
1

Flattening of a 3x3 image matrix into a 9x1 vector



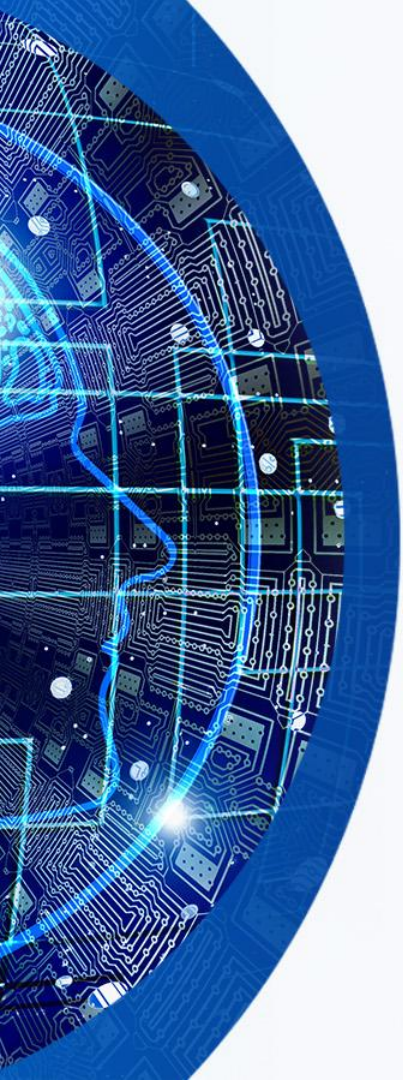
Why CNNs over Feed-Forward Neural Nets?

- Suppose an image is of the size $68 \times 68 \times 3$. The input feature dimension then becomes 13,872. This will be even bigger if we have larger images (say, of size $720 \times 720 \times 3$).
- Now, if we pass such a big input to a neural network, the number of parameters will be as well a HUGE number (depending on the number of hidden layers). This will result in **more computational and memory requirements**.
- **Before GPUs**, computers were **not able to process large amounts of image data** within a reasonable time, and the training was performed on **images with low resolutions only**.



The importance of CNN

- **CNN** has the ability **to scale for image data efficiently without increasing the complexity.**
- In contrast to feedforward neural networks, convolutional neural networks look at one patch of an image at a time and move forward in this manner to derive complete information.
- All the layers in CNNs are arranged in dimensions just like an image, including the input and output layer.
- These layers are made up of small activation units that use the principles of linear algebra (image = matrix) to extract patterns.



Applications of CNNs

- Facial recognition
- Medical imaging
- Document analysis
- Autonomous driving
- Biometric authentication
-etc.

Image Representation

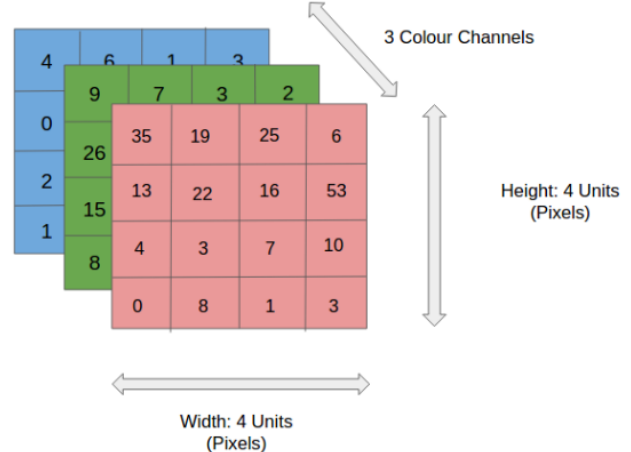
- There are definitions you should know before understanding CNN
- **Image Representation**
 - Images are encoded into colour channels, the image data is represented into each colour intensity in a colour channel at a given point, **the most common one being RGB, which means Red, Blue and Green.**



Image Representation

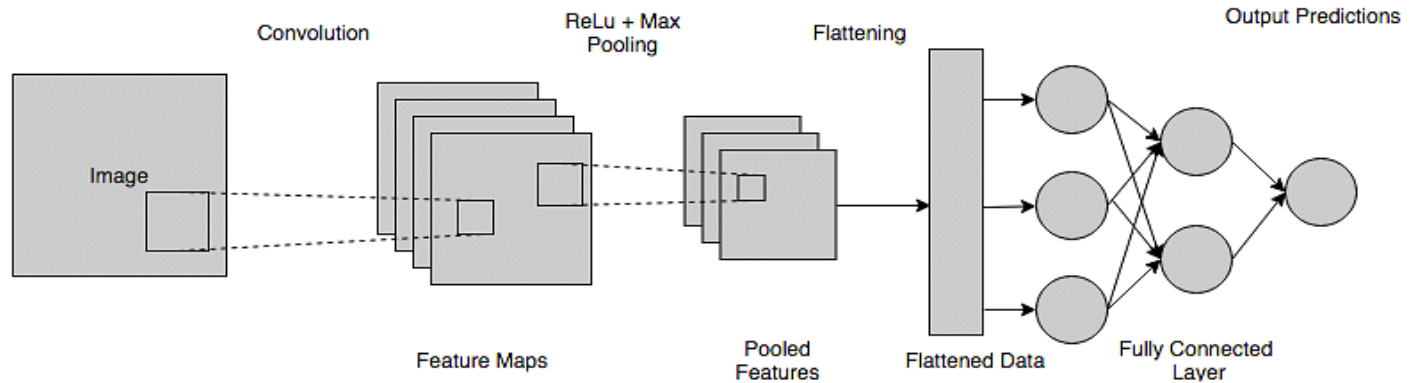
- **Image Representation:** An image is typically represented as a 3D tensor:
 - **Height:** Number of pixels in the vertical direction.
 - **Width:** Number of pixels in the horizontal direction.
 - **Channels:** Number of color channels (e.g., 3 for RGB images).

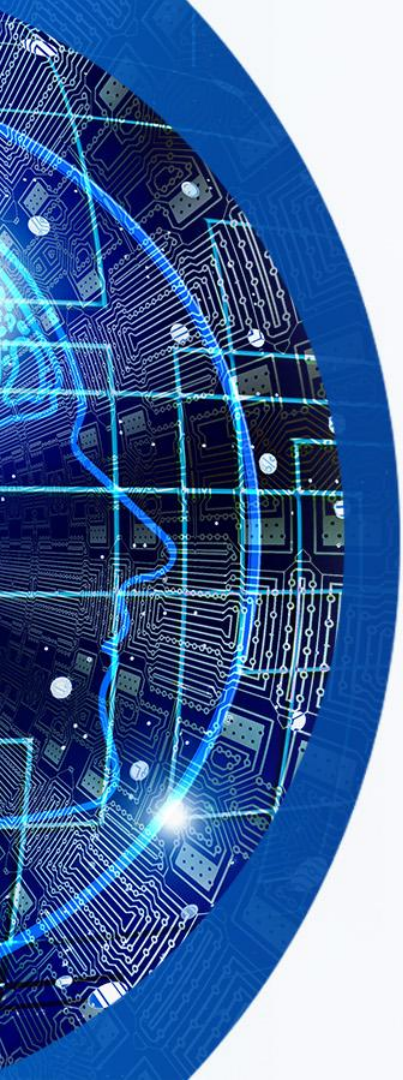
A tensor is a multidimensional array used to represent data. It's a fundamental data structure that underpins the operations within a CNN.



CNN Operations

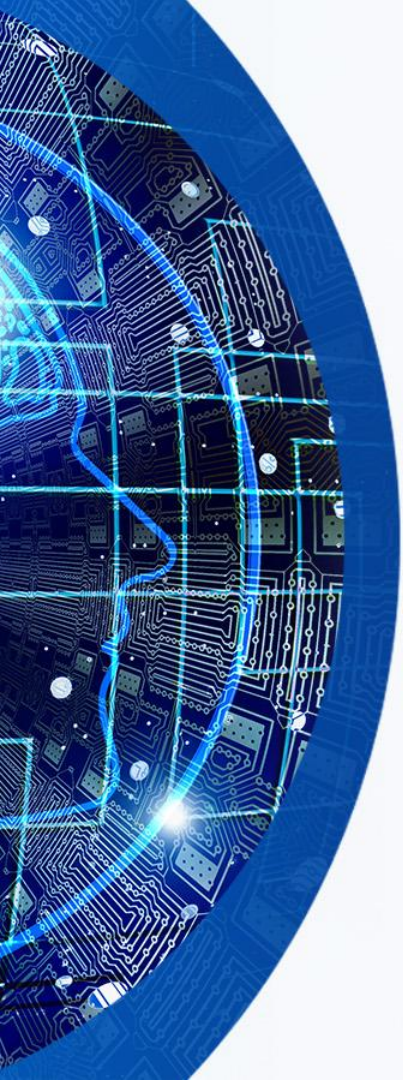
- There are certain steps/operations that are involved CNN. These can be categorized as follows:
 - Convolution
 - Activation map (ReLU)
 - Pooling layer
 - Flattening
 - Fully connected layer





The Convolution Operation

- **Convolutional Layer:** is the building block of CNN where the computation happens to **extract useful information** from image data. Each convolutional layer **consists of** learnable **filters, also known as kernels**.
- **Filters, or kernels,** are the fundamental building blocks of Convolutional Neural Networks (CNNs). They are small matrices of weights that slide across the input data, performing **element-wise multiplication and summation** to extract features.
- **Features** refer to minute details in the image data like edges, borders, shapes, textures, objects, circles, etc.
- **Filters** are designed to **extract** specific **features** from the input data, such as edges, corners, textures, or more complex patterns.

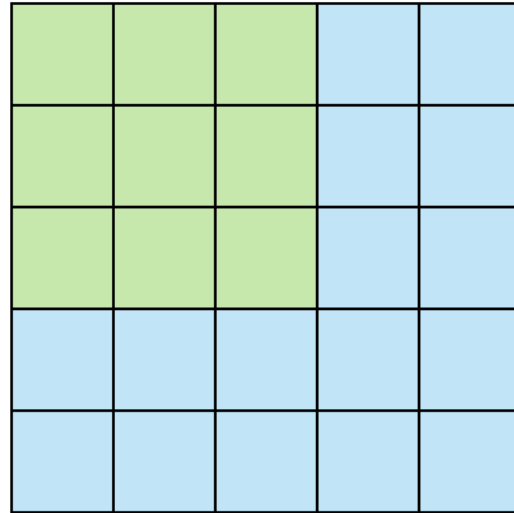


The Convolution Operation

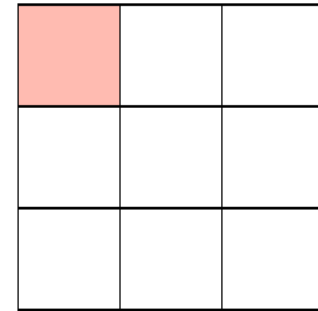
- The first few convolutional layers are responsible for capturing the **Low-Level features** such as edges, color, gradient orientation, etc.
- The deeper the network goes, the more sophisticated the **pattern searching becomes**. The architecture adapts to the **High-Level features**.
- For example, in **later layers** rather than edges and simple shapes, **filters may detect specific objects like eyes or ears**, and eventually a cat, a dog.

The Convolution Operation

- Filter (in Green Color), starting from the top-left position, scans and slides through the image from left to right across the image from top to bottom and creates a feature map.



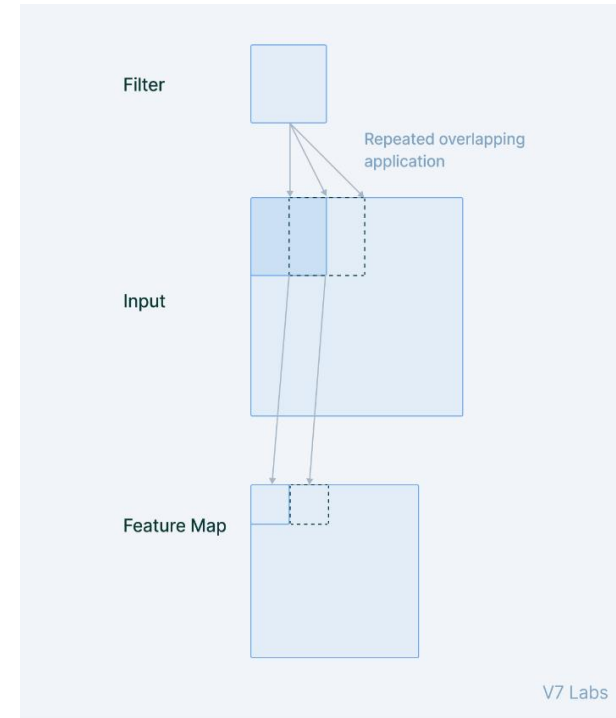
Input

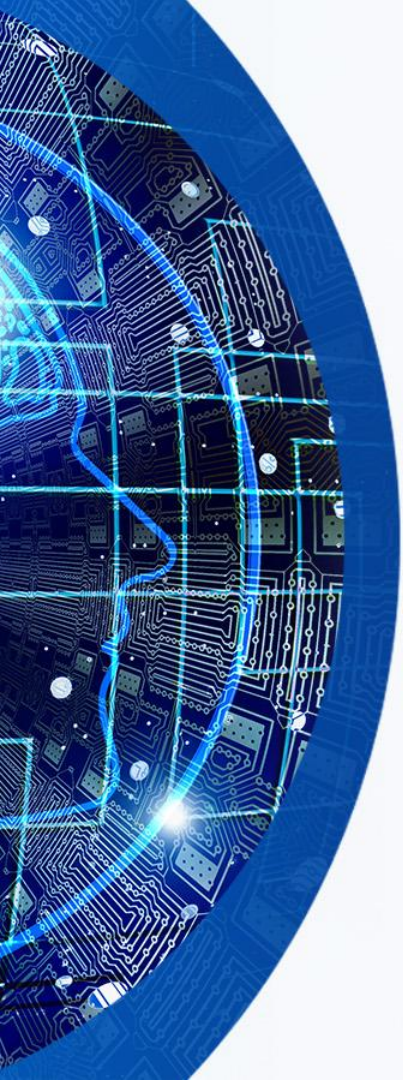


Feature Map

The Convolution Operation

- The translation of the filter to every possible position of the input matrix of the image gives an opportunity to discover that feature is present anywhere in the image.
- The generated resulting matrix is called **the feature map**.
- The feature map contains the feature information that has been extracted from the image using the specific filter.
- In the final stage of CNN, we stack all the output feature maps along with the depth and produce the output.





The Convolution Operation

The key characteristics of filters in CNNs:

- Filters are typically small, square matrices, such as 3×3 or 5×5 .
- The size of the filter determines the spatial extent of the features it can capture.
- The depth of a filter corresponds to the number of input channels. For example, in an RGB image, a filter would have a depth of 3 to process the red, green, and blue channels.
- The elements within a filter are initially random weights.
- During the training process, these weights are adjusted through backpropagation to learn the optimal patterns in the data.

The Convolution Operation

- In the convolution layer, we move the **filter/kernel** to every possible position on the input matrix. Element-wise multiplication between the filter-sized patch of the input image and filter is done, which is then summed.

Input (5 X 5)

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

Filter (3 X 3) Feature Map (3 x 3)

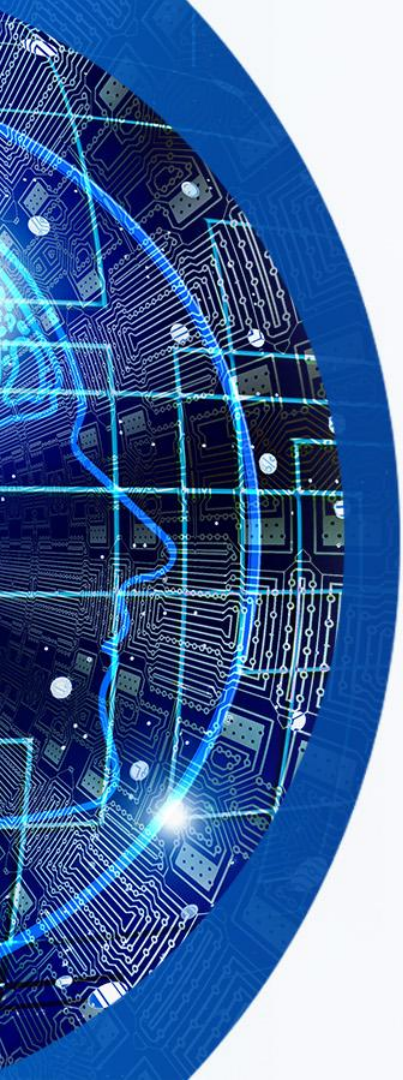
1	0	-1
1	0	-1
1	0	-1

 $*$

6		

 $=$

$$\begin{aligned} &7 \times 1 + 4 \times 1 + 3 \times 1 + \\ &2 \times 0 + 5 \times 0 + 3 \times 0 + \\ &3 \times -1 + 3 \times -1 + 2 \times -1 \\ &= 6 \end{aligned}$$



The Convolution Operation

- If we have $N \times N$ image size and $F \times F$ filter size then after convolution result (feature map) dimension will be

$$N - F + 1$$

- If image dimension is 4×4 and filter is 3×3 , hence we are getting output after convolution is 2×2 .

The Convolution Operation

- In the below demonstration, the green section resembles our 5x5x1 input image. The element involved in carrying out the convolution operation in the first part of a Convolutional Layer is called the Kernel/Filter, K, represented yellow colour. We have selected K as a 3x3x1 matrix.

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

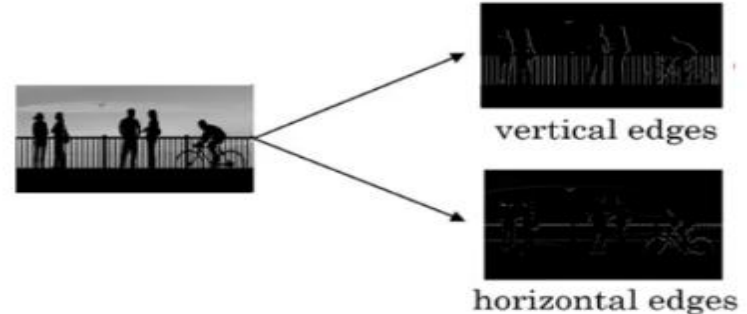
Convolved
Feature

Edge Detection Example

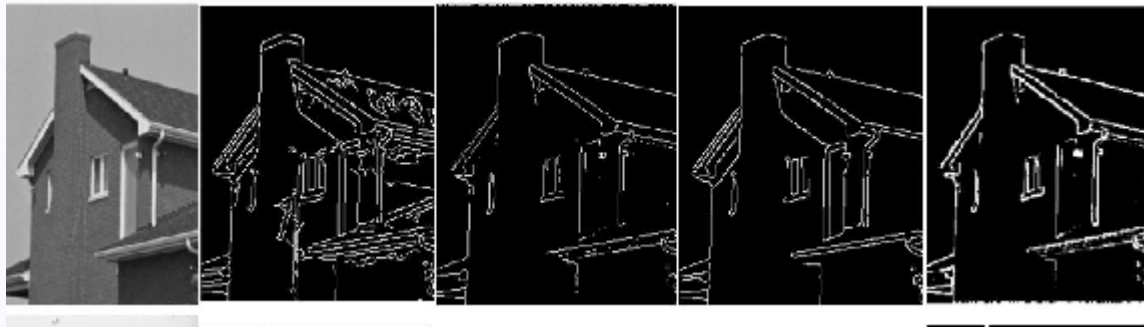
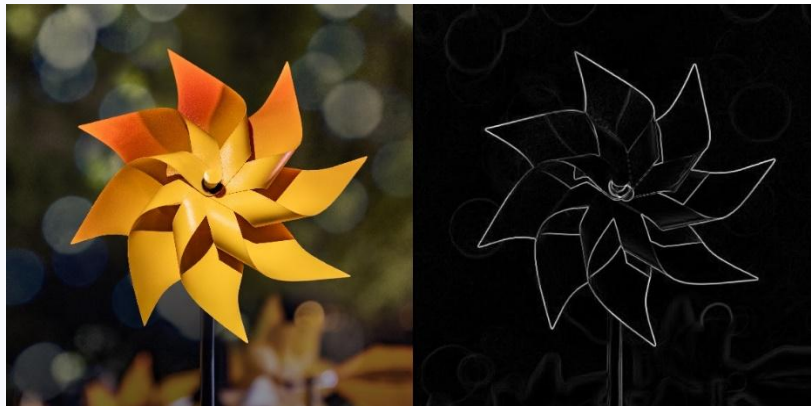
- We will focus on how the edges can be detected from an image. Suppose we are given the below image:



- Every image has vertical and horizontal edges which combining to form an image. As you can see, there are many vertical and horizontal edges in the image. The first thing to do is to detect these edges:



Edge Detection Example



Edge Detection Example

- Consider one more example:

Note: Higher pixel values represent the brighter portion of the image, and the lower pixel values represent the darker portions.

10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

Vertical

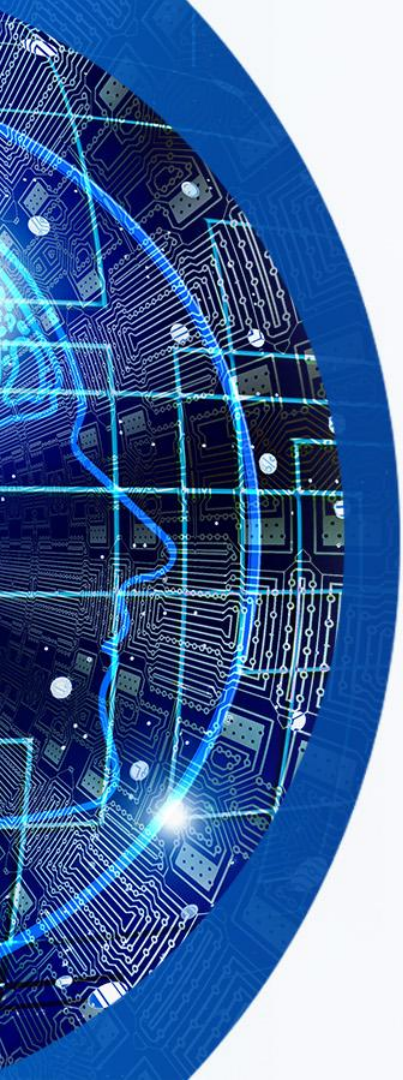
=

0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0



- This is an example of a vertical edge detector because it can detect the sharp edge in the middle of the original image, shown by the brighter region in the centre of the resulting image.

Edge Detection Example



10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

Vertical

=

0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0
0	0	30	30	0	0

sharp change in
colours or intensity.

- Applying the convolution operator between the 3×3 filter (center) and the blue region of the original image (left) gives rise to the element in the new image in blue box (right), whose value can be calculated as the sum of the resulting element-wise product between the 2 matrices: $(10*1)+(10*0)+(10*(-1))+(10*1)+(10*0)+(10*(-1))+(10*1)+(10*0)+(10*(-1)) = 0$

More Edge Detection

- The non-zero band in the middle reveals the horizontal edge in the center of the original image but also discovers the change from bright to dark (left side) and dark to bright (right side).

10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
10	10	10	10	0	0	0	0
0	0	0	0	10	10	10	10
0	0	0	0	10	10	10	10
0	0	0	0	10	10	10	10
0	0	0	0	10	10	10	10

*

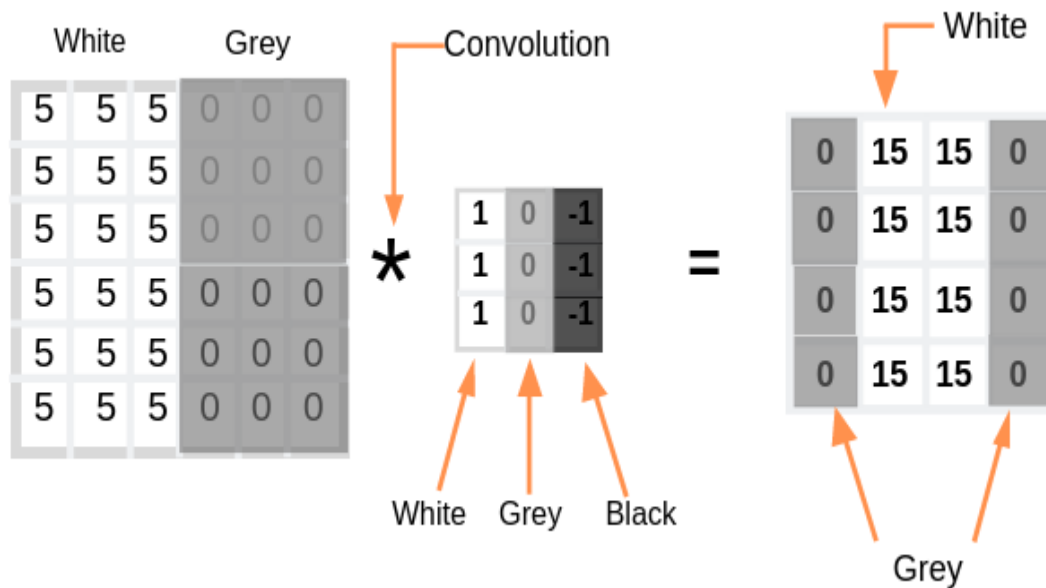
1	1	1
0	0	0
-1	-1	-1

Horizontal

=

0	0	0	0	0	0
0	0	0	0	0	0
30	30	10	-10	-30	-30
30	30	10	-10	-30	-30
0	0	0	0	0	0
0	0	0	0	0	0

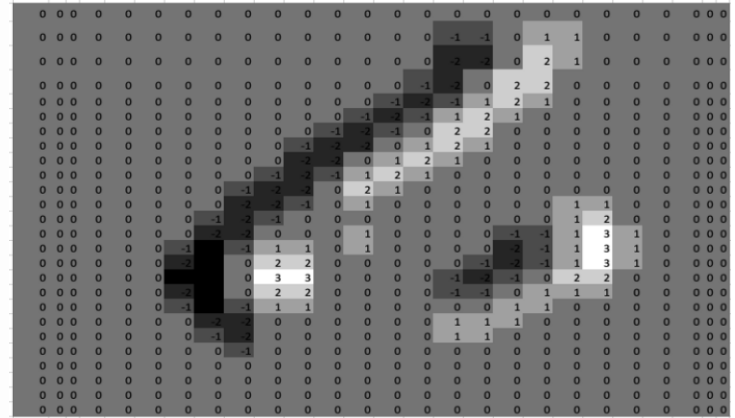
An example of horizontal edge detection



[illegible]

- # More Edge Detection
- Using another example (for the digit “6”): we can detect all the vertical edges with using this filter:
-
- | | | |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

1	0	-1
1	0	-1
1	0	-1



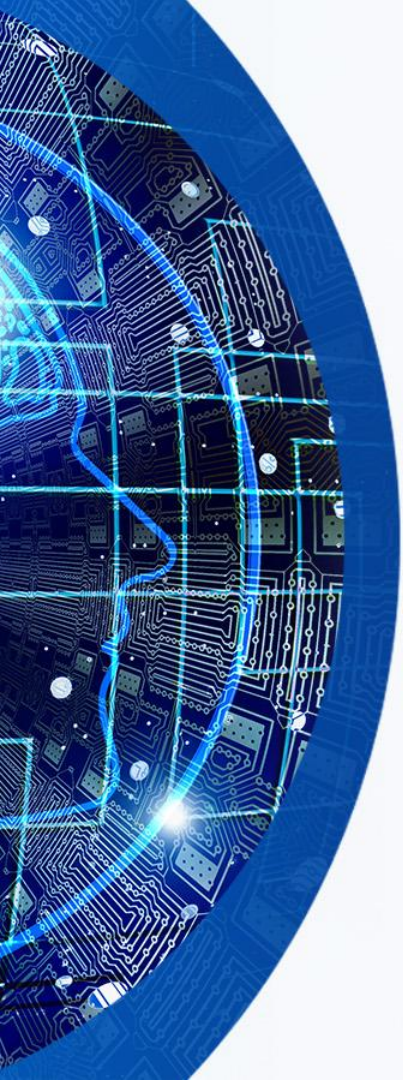
[illegible]

- We can also detect horizontal edges with this filter:

[illegible]

-1	-1	-1
0	0	0
1	1	1

[illegible]



Hyperparameters of Convolutional Layer

- Convolution
 - Filter Size
 - Number of Filters
 - Padding
 - Stride

Padding

- There are primarily two disadvantages of convolutional operation:
 - Every time we apply a convolutional operation, the size of the image shrinks. The dimensions of the feature maps would diminish in deep CNNs
 - Pixels present in the corner of the image are used only a few number of times during convolution as compared to the central pixels. Hence, we do not focus too much on the corners since that can lead to information loss

Corners

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

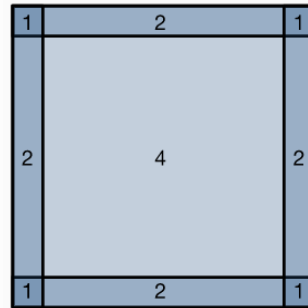


-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Shrink

Padding

- The pixels in the corners are hardly used at all. This figure shows number of times the convolution utilizes the pixels.

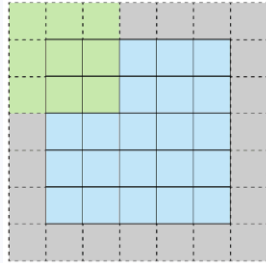


- To overcome these issues, we can pad the image with an additional border, i.e., we add one pixel all around the edges (zero-padding).

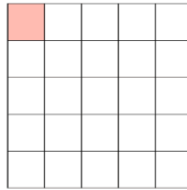
0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

Padding

- We use padding to make sure that convolutional layers do not unnecessarily decrease the height and width of the feature map due to filters. Padding is a process of adding zeros to the input matrix symmetrically.

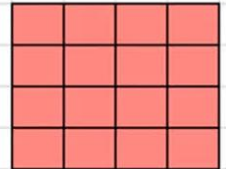
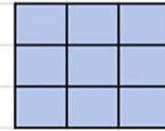


Stride 1 with Padding



Feature Map

0	0	0	0	0	0
0	0.3	0.5	0.9	1.0	0
0	1.0	1.0	1.0	1.0	0
0	0.9	0.9	0.5	0.3	0
0	0.2	0.0	0.0	0.0	0
0	0	0	0	0	0



Input
4 x 4

Filter
3 x 3

Output
4 x 4

Padding

- This means that the input will be a 7 X 7 matrix (instead of a 5 X 5 matrix). Applying convolution of 3 X 3 filter on it will result in a 5 X 5 matrix which is the original shape of the image.

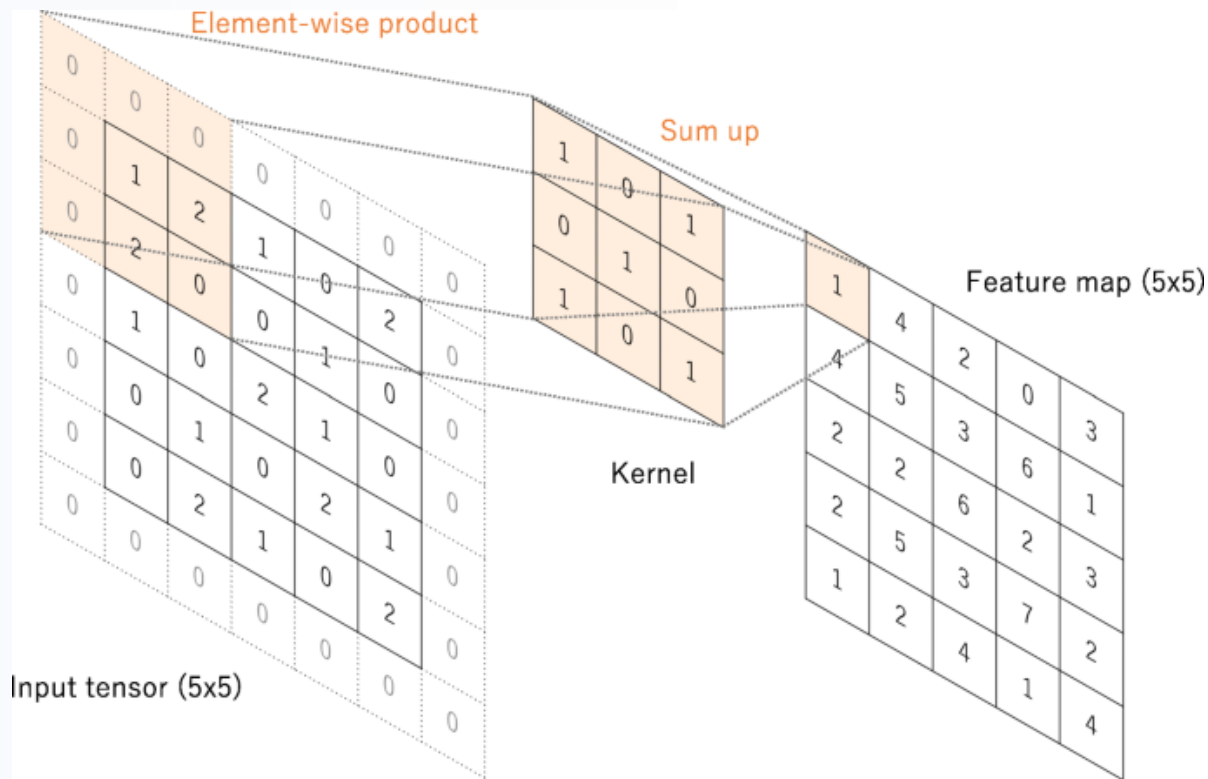
0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

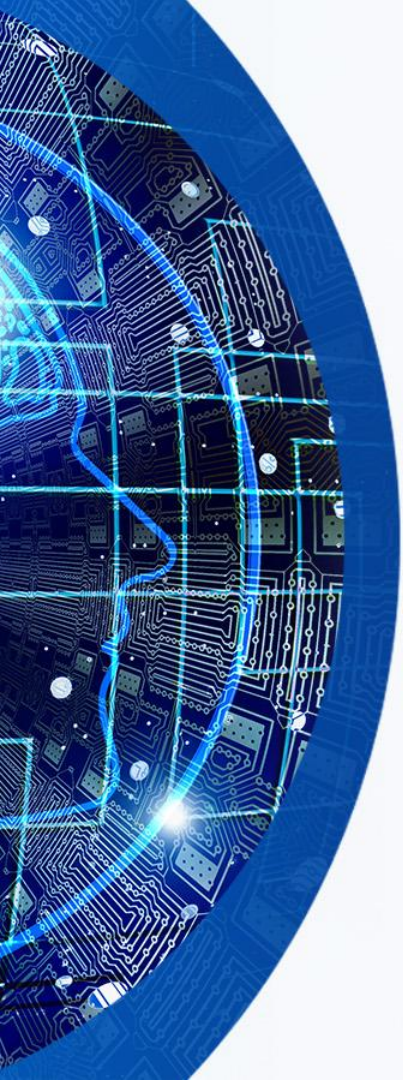
Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Padding





Padding

- **Same Padding**
- How many pixels you use for same padding depends entirely on **the size of the filter**. The most used filter sizes are 3×3, 5×5, and 7×7. We calculate the required padding using the following equation:

$$padding = \frac{f - 1}{2}$$

- **Example**

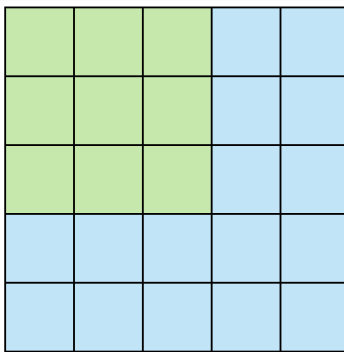
- In the case of a **3×3 filter**, **one layer of zeros** must be added to the borders. ($padding = (3 - 1) / 2 = 1$)
- If we had a **5×5 filter**, we would have to pad each edge with **two layers** of zeros to the border of the image ($padding = (5 - 1) / 2 = 2$)

0	-1	0			
-1	5	-1	0	0	
0	-1	0	0	0	
25 5	25 5	25 5	0	0	0
25 5	25 5	25 5	0	0	0
25 5	25 5	25 5	0	0	0
25 5	25 5	25 5	0	0	0

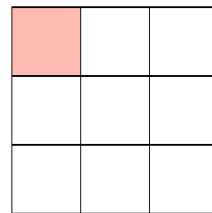
2	2	4	2	2				
1	1	2	1	1				
0	0	0	0	0	0	0	0	
-1	-1	-2	-1	-1	0	0	0	
-2	-2	-4	-2	-2	0	0	0	
		25 5	25 5	25 5	0	0	0	
		25 5	25 5	25 5	0	0	0	
		25 5	25 5	25 5	0	0	0	

Stride

- Stride refers to the number of pixels we slide while matching the filter with the input image patch
- Stride denotes how many steps we are moving in each steps in convolution. By default, it is one.



Stride 1

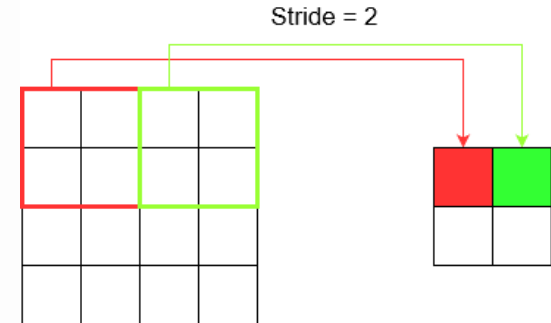
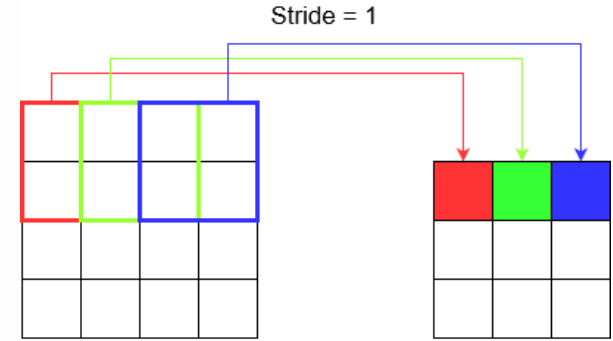


Feature Map

Stride

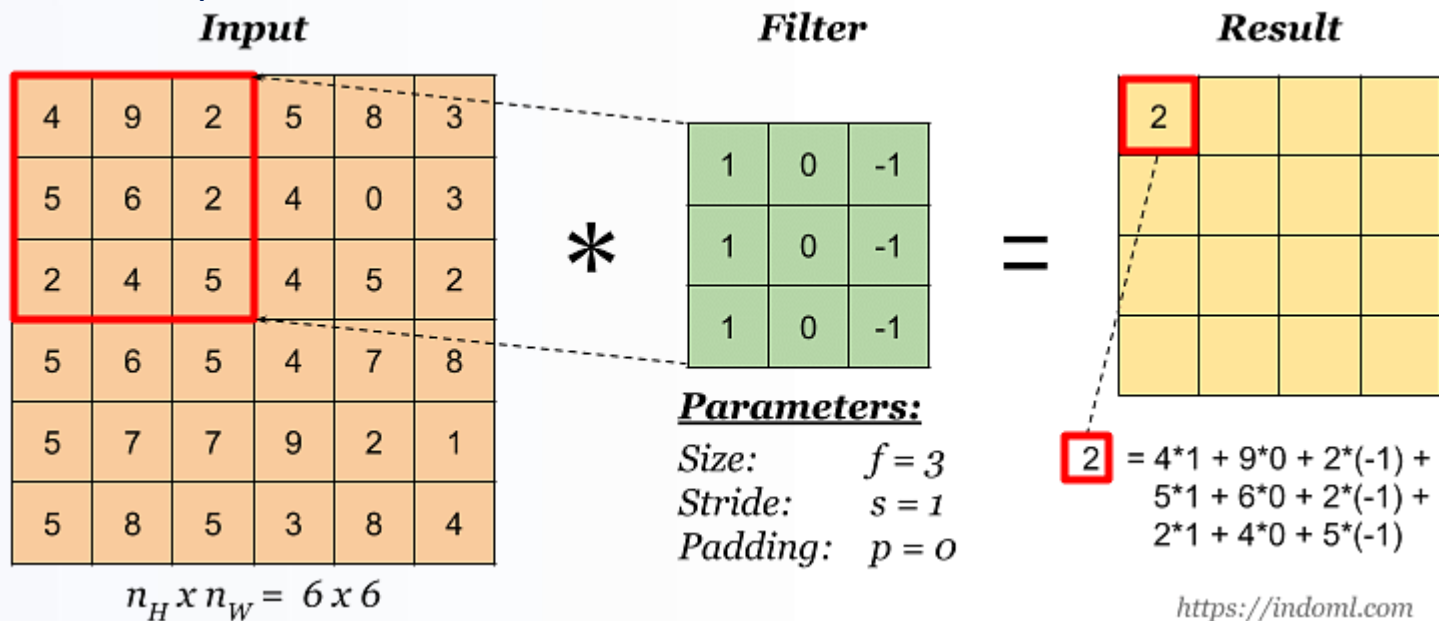
- We can calculate the length of the output feature map o depending on the filter length size f , the length of the original image n and the stride s as follows.

$$o = \frac{n - f}{s} + 1$$



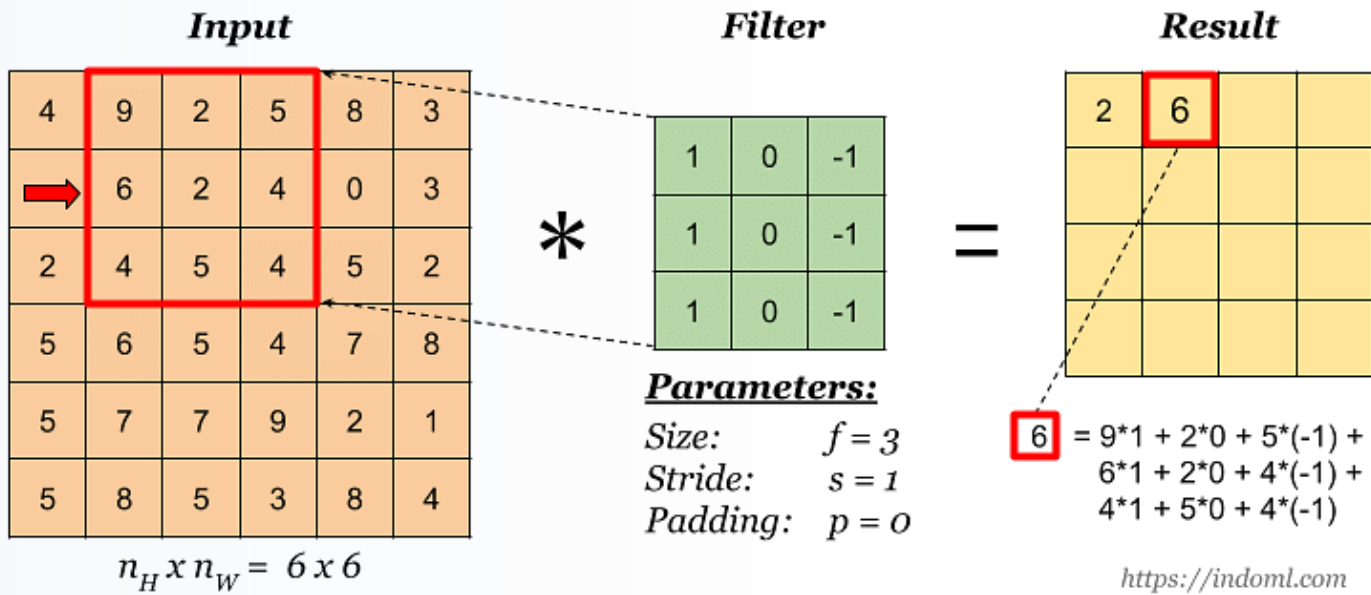
Stride

- The filter here moves by 1 pixel as the stride was taken in the following example is 1, and the dot product calculation is repeated.



Stride

- It will continue similarly moving across the entire matrix and generate a feature map. The stride's value is any integer, and a larger stride results in a smaller output matrix.

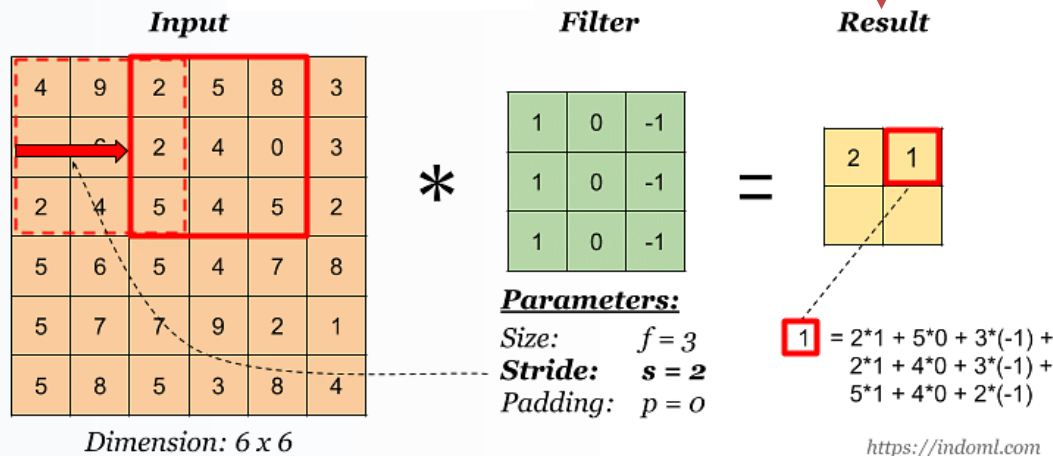


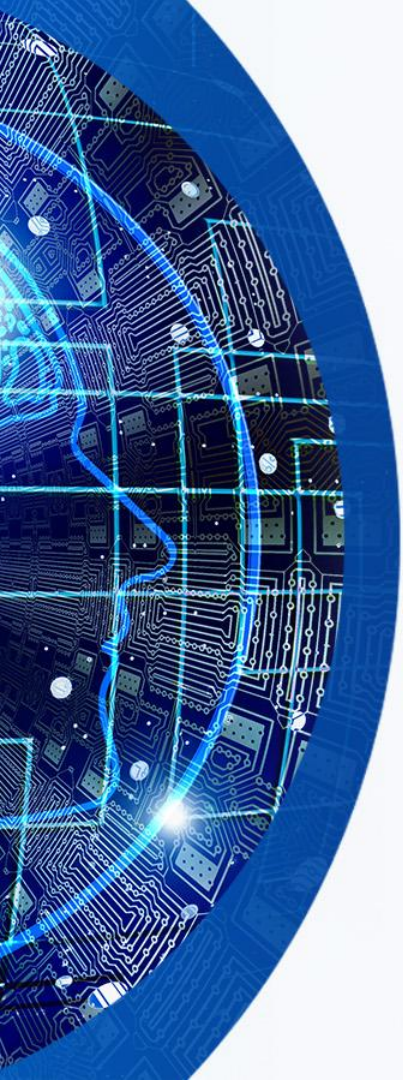
Stride

- It might also happen that the filter does not entirely fit the input image matrix, which means as the filter moves across the image matrix, some aspects of the filter fall outside the image. For example, consider the example given below with Stride = 2.
- Use this equation:

$$o = \frac{n - f}{s} + 1$$

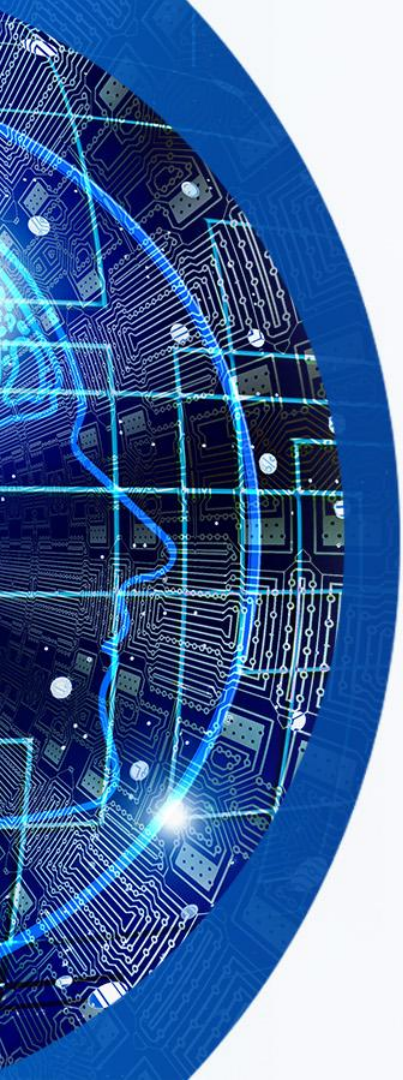
output dimension (feature map)
 $= ((6-3)/2) + 1 = 2.5 \approx 2$





Stride

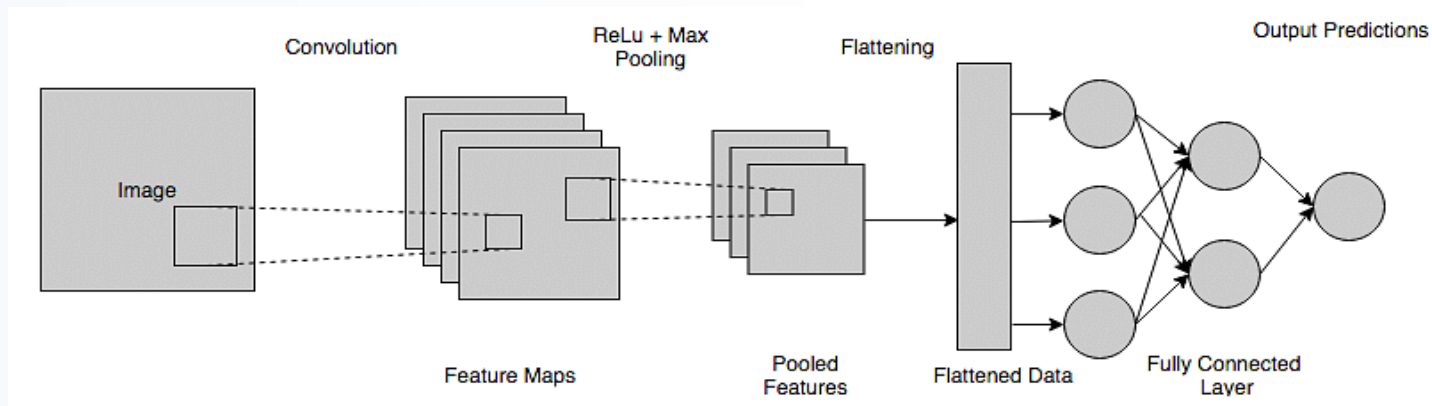
- **Why Do We Need Strided Convolutions?**
 - **Reducing Computational Cost:** By increasing the stride, we can reduce the number of operations required to process the input data.
 - **Improving Efficiency:** Strided convolutions allow us to gradually reduce the spatial dimensions of feature maps as we go deeper into the network. Smaller feature maps require less memory and computational resources, making the model more efficient to train and deploy.

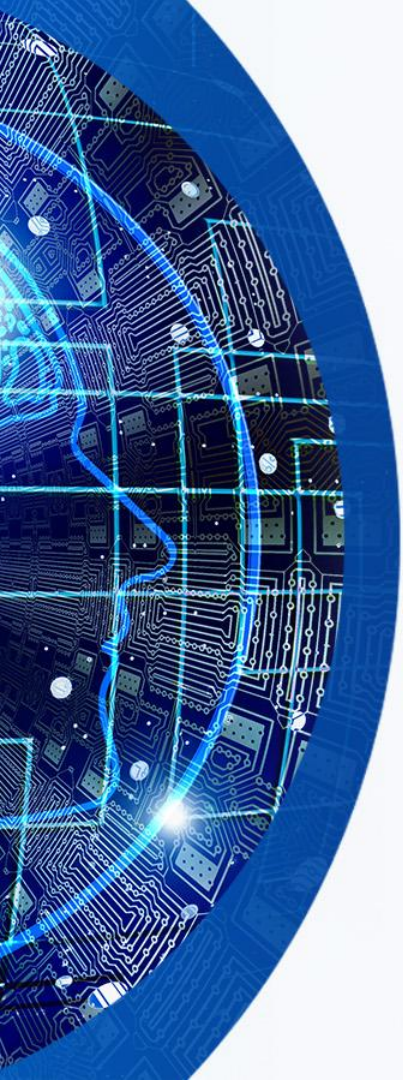


Stride

- **While strided convolutions are a powerful tool in CNNs, they also have some potential drawbacks:**
 - If the stride is too large, the model may skip over important features in the input data. This can result in a significant loss of information and reduced performance.
- **Conclusion:** If our neural network only requires an understanding of higher-level features, we can make the learning process computationally more efficient by choosing a larger stride.

CNN Operations



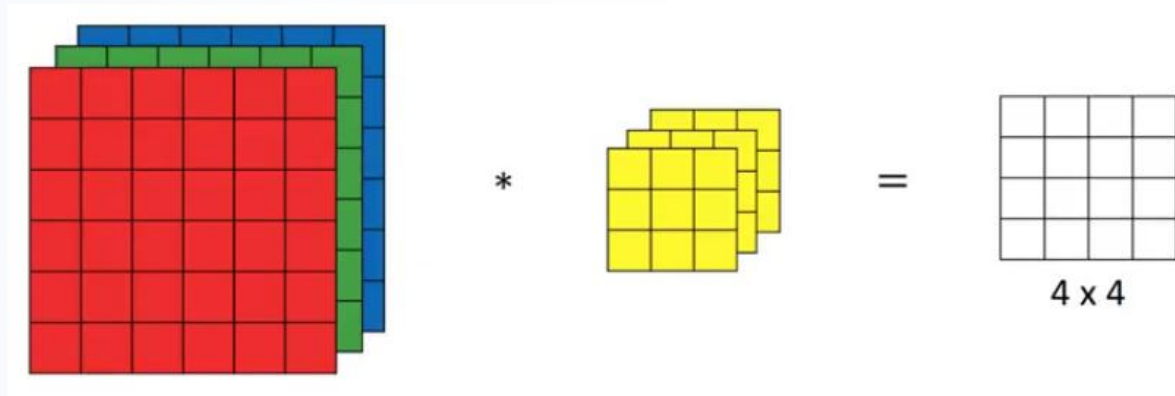


Convolution Operation on Volume

- Suppose, instead of a 2-D image, we have a 3-D input image of shape $6 \times 6 \times 3$. How will we apply convolution on this image? We will use a $3 \times 3 \times 3$ filter instead of a 3×3 filter. Let's look at an example:
- **Input: $6 \times 6 \times 3$**
- **Filter: $3 \times 3 \times 3$**

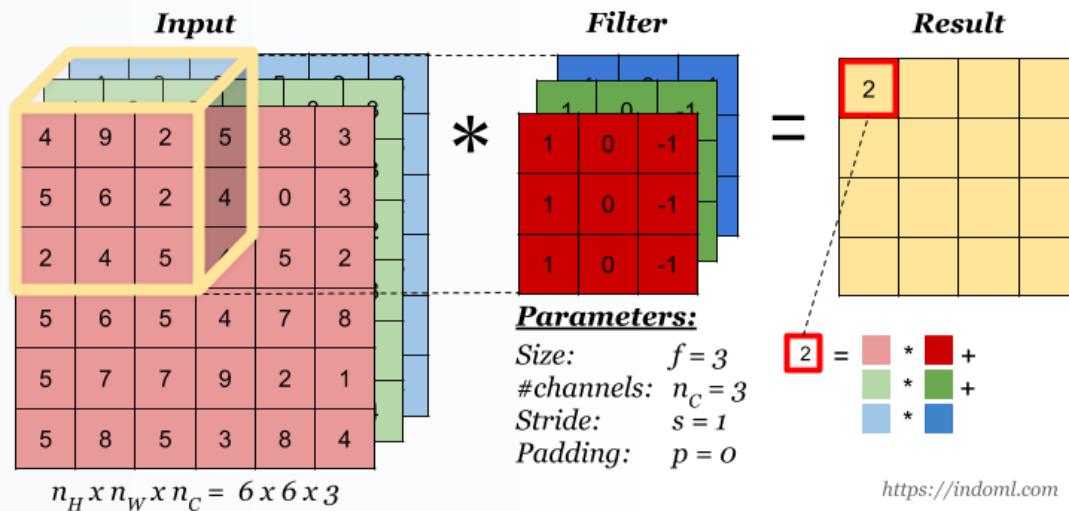
Convolution Operation on Volume

- **Input: 6 X 6 X 3**
- **Filter: 3 X 3 X 3**
- The dimensions above represent the height, width and channels in the input and filter. Keep in mind that the number of channels in the input and filter should be same. This will result in an output of 4 X 4. Let's understand it visually:



Convolution Operation on Volume

- To calculate one output cell, perform convolution on each matching channel, then add the result together.



Convolution Operation on Volume

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2

0	1	1
0	1	0
1	-1	1

Kernel Channel #3

308

+

-498

+

164

+ 1 = -25

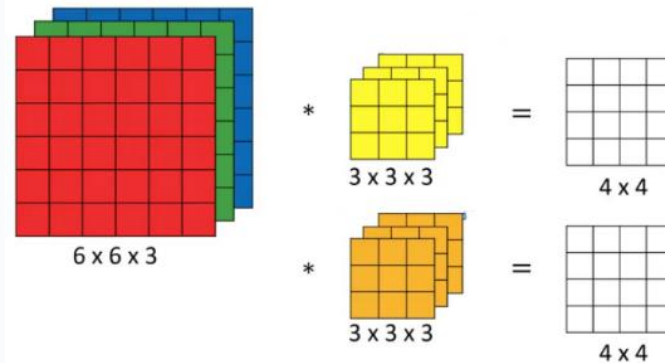
↑
Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

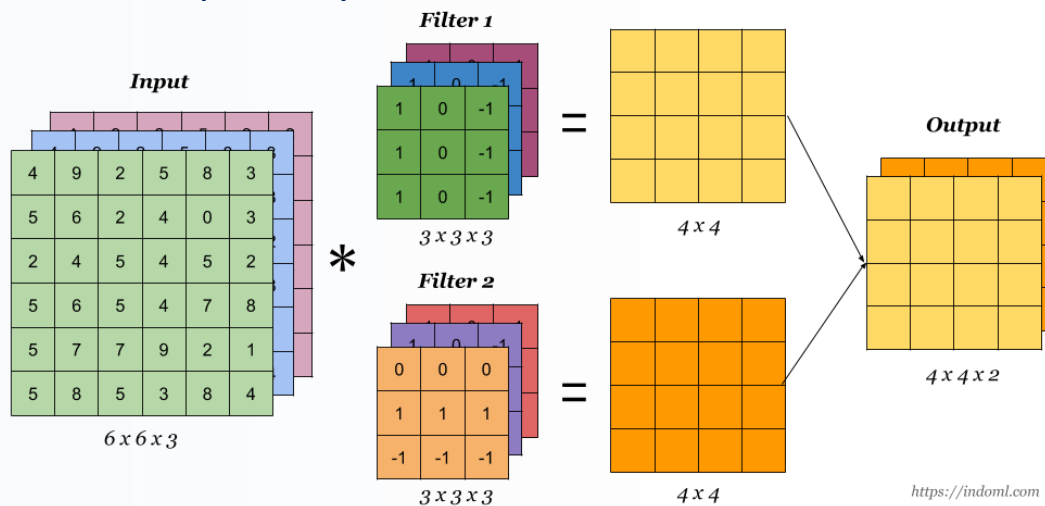
Convolution Operation with Multiple Filters

- Instead of using just a single filter, **we can use multiple filters** as well. How do we do that? Let's say **the first filter will detect vertical edges** and **the second filter will detect horizontal edges** from the image. If we use multiple filters, the output dimension will change. So, instead of having a 4 X 4 output as in the previous example, **we would have a 4 X 4 X 2 output (if we have used 2 filters):**



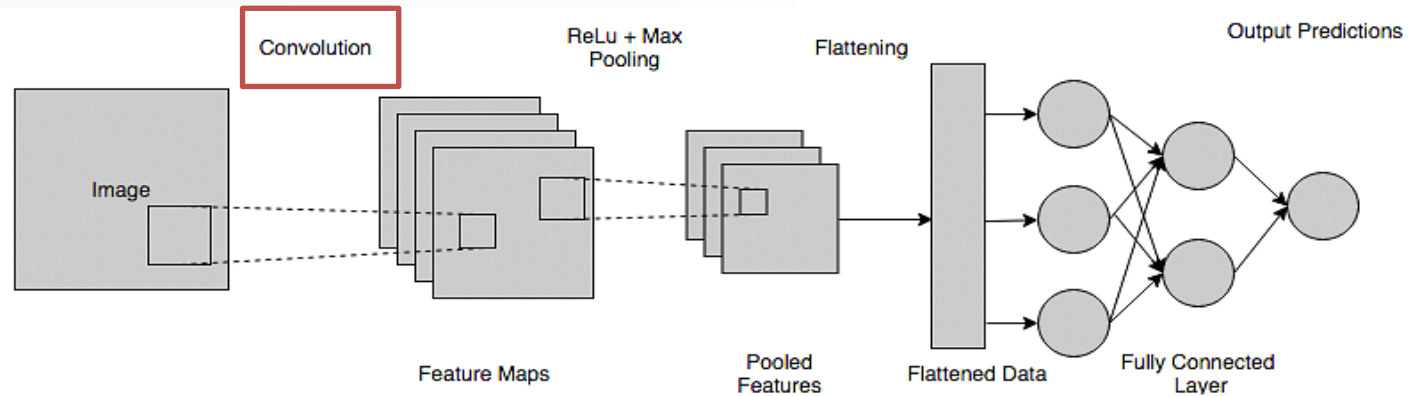
Convolution Operation with Multiple Filters

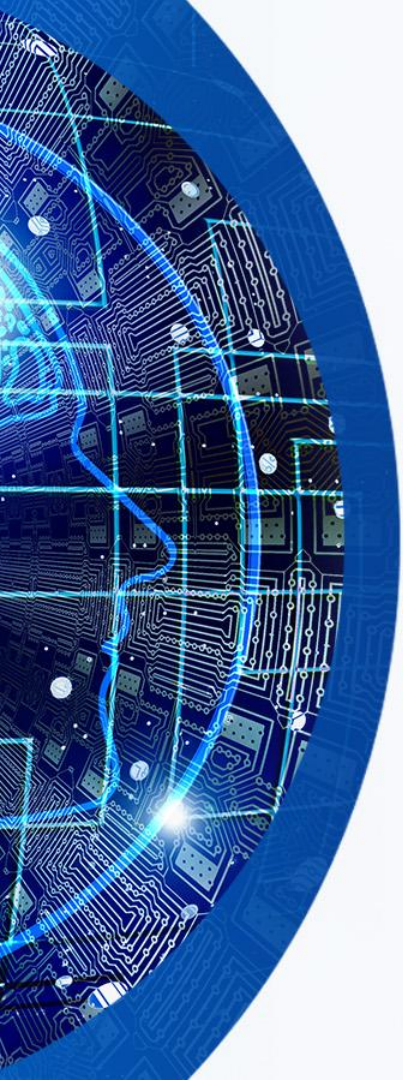
- Example:** the input data is of size $6 \times 6 \times 3$. The number of units (filters) is 2, each of which has the dimensions $3 \times 3 \times 3$. The output is $4 \times 4 \times 2$. So, in general channels need to be treated separately under each filter.



Summary: CNN Operations

- Convolution
 - Filter Size ✓
 - Number of Filters ✓
 - Padding ✓
 - Stride ✓





Thank You